МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Донецкий национальный технический университет»

Факультет интеллектуальных систем и программирования
Кафедра «Программная инженерия» им. Л.П. Фельдмана

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
по дисциплине «Программирование систем с серверами баз данных»

Тема:
«Создание клиент-серверной информационной системы средствами СУБД»
Вариант Б-16: учёт услуг связи интернет-провайдеров

Руководители:
ст. преп. каф. ПИ Щедрин С.В.
асс. каф. ПИ Филипишин Д. А.

Разработал:
ст. гр. ПИ-24б
Ольховский Д. Д.

Донецк – 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНЕЦКИЙ НАЦИОНАЛЬНЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет Интеллектуальных систем и программирования

Кафедра «Программная инженерия» им. Л.П. Фельдмана

Утверждаю
зав. кафедрой ПИ им. Л.П. Фельдмана
проф. Зори С.А.

_____
__. __.2026 г.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
на курсовой проект по дисциплине
«Программирование систем с серверами баз данных»

Тема: «Создание клиент-серверной информационной системы средствами СУБД»

выдано студенту группы ПИ-24б Ольховскому Дмитрию Дмитриевичу

Руководители курсового проекта: ст. пр. Щедрин С.В., асс. Филипишин Д. А.

Описание предметной области:

16. Для автоматизации учета предоставления услуг связи фирмами-провайдерами Internet необходима информация о фирмах-провайдерах (название, тип собственности (государственный, частный, ЗАО, ОАО,…), адрес, телефон, год начала работы), о заключенных с ними договорах (абонент (ФИО или название предприятия, тип (частное лицо, школа, ВУЗ, предприятие, …), физический адрес, IP-адрес, телефон), дата подключения, стоимость подключения, стоимость 1 Мб информации) и о сеансах модемной связи (дата и время сеанса, количество Мб, тариф (бизнес-время — 100% оплата, вечерний тариф — скидка 20%, ночной тариф — скидка 50%,…)).

Примечание: один абонент может заключать несколько договоров.

Донецк – 2026

Задание на курсовой проект

1. Спроектировать концептуальную модель базы данных (БД) для заданной предметной области и представить ее в виде взаимосвязанных таблиц, находящихся в третьей нормальной форме (в случае денормализации БД – обосновать необходимость). Выделить базовые таблицы и таблицы-справочники, указать для них первичные и внешние ключи.
2. Создать базу данных в среде СУБД средствами языка SQL. Добавить таблицы, домены, индексы.
3. Разработать не менее шести триггеров (по одному для каждого типа события), как минимум для двух различных таблиц БД.
4. Заполнить таблицы БД с использованием соответствующих запросов на языке SQL (не менее десяти записей в каждом справочнике, не менее 10 000 - 50 000 псевдослучайных записей в таблицах).
5. Сформулировать следующие виды запросов:
   - симметричное внутреннее соединение с условием (два запроса с условием отбора по внешнему ключу, два – по датам);
   - симметричное внутреннее соединение без условия (три запроса);
   - левое внешнее соединение;
   - правое внешнее соединение;
   - запрос на запросе по принципу левого соединения;
   - итоговый запрос без условия;
   - итоговый запрос без условия с итоговыми данными вида: «всего», «в том числе»;
   - итоговые запросы с условием на данные (по значению, по маске, с использованием индекса, без использования индекса);
   - итоговый запрос с условием на группы;
   - итоговый запрос с условием на данные и на группы;
   - запрос на запросе по принципу итогового запроса;
   - запрос с использованием объединения

- запросы с подзапросами (с использованием in, not in, case, операциями над итоговыми данными).

6. Запросы без параметров реализовать в виде представлений, остальные запросы – в виде хранимых процедур и/или функций. Создать, по меньшей мере, одно модифицируемое представление, используя механизм триггеров. ВСЯ логика проектируемого ПО – на сервере.

7. Разработать клиентское приложение, которое предоставляет следующие возможности для работы с созданной базой данных: многопользовательский режим работы (одна программа для всех ролей – ситуативный доступ к интерфейсу)

- наличие нескольких ролей пользователя (администратор – добавление/удаление/редактирование пользователей, их прав/ролей; пользователи_1 – …, пользователи _2 – ...)
- просмотр содержимого таблиц и представлений (здесь и далее – с учетом прав пользователей);
- добавление, редактирование и удаление записей таблиц и модифицируемых представлений;
- работа с наборами данных, находящимися в отношении «один-ко-многим» (создать составную форму для просмотра и редактирования данных родительской и дочерней таблиц);
- поиск и фильтрация данных отображаемых таблиц;
- просмотр результатов выполнения запросов;
- визуализация результатов одного из итоговых запросов (диаграммы, экспорт в Excel).

8. Обеспечить защиту данных, информации от несанкционированного доступа, сделать защиту на уровне строк, выполнить партицирование одной из основных таблиц

Рекомендуемое содержание пояснительной записки:

Титульный лист

Техническое задание

Реферат

Содержание

Введение

1. Описание предметной области, постановка задачи

2. Обоснование выбора СУБД, описание возможностей СУБД

3. Обоснование выбора инструментальные средств для написания клиентской части, проектирование структуры ПО

3.1 Невизуальные компоненты для работы с данными

3.2 Визуальные компоненты отображения данных

3.3 Разработка шаблонов приложений для работы с таблицами базы данных

4. Проектирование базы данных в выбранной СУБД

4.1 Проектирование концептуальной модели БД

4.2 Создание таблиц, доменов, индексов, сиквенсов

4.3 Разработка триггеров

4.4 Организация многоролевого доступа к данным

4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина)

4.6 Партицирование одной из основных таблиц БД

4.7 Проектирование запросов к базе данных

4.8 Создание представлений и хранимых процедур, функций

5. Разработка клиентского приложения

5.1 Формы и компоненты для работы в «роли 1»

5.2 Формы и компоненты для работы в «роли 2»

5.3 …

5.4 Генерация результатов не менее трех итоговых запросов (диаграммы, экспорт в Excel)

6 Тестирование разработанной информационной системы (в т.ч. включая защиту от несанкционированного доступа, одновременную работы с данными, каскадное удаление)

Заключение/выводы и предложения

Список литературы

Приложение А. Листинг шаблонов

Приложение Б. Листинг серверного приложения

Приложение В. Листинг клиентского приложения

Приложение Г. Руководство пользователя

Приложение Д. Руководство суперпользователя

Приложение Е. Руководство администратора

График выполнения курсового проекта

Неделя	Работа
1-2	Выдача и изучение задания
3	Анализ требований к системе и способов их реализации
4-5	Проектирование и реализация БД (таблицы, домены, индексы, роли, RLS, партицирование)
6-7	Создание триггеров и заполнение таблиц БД
8-9	Создание представлений и хранимых процедур, запросов
10-13	Разработка клиентского приложения
14	Тестирование и отладка системы
15	Оформление пояснительной записки
16-17	Защита курсового проекта

Дата выдачи задания: 02.09.2026

Дата защиты курсового проекта: 23.12.2026

Задание принял: Ольховский Д.Д. _____

(подпись студента)

Согласовано:

Руководители проекта Щедрин Сергей Валерьевич

Филипишин Дмитрий Александрович

РЕФЕРАТ

Пояснительная записка содержит 88 с., 77 рис., 8 табл., 4 источников, 6 прил.

Объект разработки — учёт услуг связи, оказываемых фирмами-провайдерами Internet: провайдеры, общий реестр абонентов, договоры, сеансы связи и тарифы.

Цель работы — создание клиент-серверной информационной системы, в которой хранение данных, проверка ограничений, расчёты, отчёты и разграничение доступа выполняются средствами СУБД PostgreSQL, а настольное приложение служит тонким клиентом.

В процессе работы построена концептуальная модель предметной области в третьей нормальной форме, создана база данных с доменами, индексами и последовательностями, заполненная псевдослучайными данными, таблицы сеансов и договоров секционированы, разработаны представления и функции схемы api для запросов на выборку всех основных видов — соединений, итоговых запросов, объединения и запросов с подзапросами, реализованы пять групп пользователей средствами PostgreSQL, ограничение видимости строк по провайдеру пользователя и сброс пароля с временным паролем. Разработано клиентское приложение на Qt 6, которое строит интерфейс по группе пользователя, выводит списки постранично, показывает отчёты с диаграммами и сохраняет их в Excel. Корректность серверной части подтверждена набором из 140 автоматических тестов.

Использованные средства: PostgreSQL 18, PL/pgSQL, pgAdmin 4, Qt 6, C++17, CMake, Qt Charts, QXlsx.

СУБД POSTGRESQL, КЛИЕНТ-СЕРВЕР, РЕЛЯЦИОННАЯ МОДЕЛЬ, НОРМАЛИЗАЦИЯ, РОЛИ, РАЗГРАНИЧЕНИЕ ДОСТУПА, ХРАНИМЫЕ ФУНКЦИИ, ПРЕДСТАВЛЕНИЯ, QT, ИНТЕРНЕТ-ПРОВАЙДЕР

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....11
1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ.....13
1.1 Описание предметной области.....13
1.2 Постановка задачи.....14
2 ОБОСНОВАНИЕ ВЫБОРА СУБД, ОПИСАНИЕ ВОЗМОЖНОСТЕЙ СУБД 16
3 ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТАЛЬНЫХ СРЕДСТВ ДЛЯ НАПИСАНИЯ КЛИЕНТСКОЙ ЧАСТИ, ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПО.....19
3.1 Невизуальные компоненты для работы с данными.....20
3.2 Визуальные компоненты отображения данных.....21
3.3 Разработка шаблонов приложений для работы с таблицами базы данных 23
4 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ В ВЫБРАННОЙ СУБД.....24
4.1 Проектирование концептуальной модели БД.....24
4.2 Создание таблиц, доменов, индексов, сиквенсов.....26
4.3 Разработка триггеров.....28
4.4 Организация многоролевого доступа к данным.....29
4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина).....32
4.6 Партицирование одной из основных таблиц БД.....33
4.7 Проектирование запросов к базе данных.....36
4.7.1 Симметричное внутреннее соединение с условием.....36
4.7.2 Симметричное внутреннее соединение без условия.....39
4.7.3 Внешние соединения и запрос на запросе.....41
4.7.4 Итоговые запросы, запрос на запросе и запрос с подзапросом.....44
4.7.5 Итоговые запросы с итогами «всего» и «в том числе», по маске и с использованием индекса.....48

4.7.6 Объединение и запросы с подзапросами.....52
4.8 Создание представлений и хранимых процедур, функций.....55
5 РАЗРАБОТКА КЛИЕНТСКОГО ПРИЛОЖЕНИЯ.....58
5.1 Формы и компоненты для работы в роли «Оператор».....58
5.2 Формы и компоненты для работы в роли «Руководитель».....60
5.3 Формы и компоненты для работы в роли «Главный аналитик».....61
5.4 Формы и компоненты для работы в роли «Администратор приложения»61
5.5 Формы и компоненты для работы в роли «Администратор БД».....63
5.6 Генерация результатов итоговых запросов (диаграммы, экспорт в Excel)65
6 ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ 66
6.1 Защита от несанкционированного доступа.....67
6.2 Одновременная работа с данными.....68
6.3 Каскадное удаление.....68
6.4 Корректность запросов и отчётов.....68
ЗАКЛЮЧЕНИЕ.....70
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....72
ПРИЛОЖЕНИЕ А. Листинг шаблонов.....73
ПРИЛОЖЕНИЕ Б. Листинг серверного приложения.....75
ПРИЛОЖЕНИЕ В. Листинг клиентского приложения.....79
ПРИЛОЖЕНИЕ Г. Руководство пользователя.....82
ПРИЛОЖЕНИЕ Д. Руководство суперпользователя.....85
ПРИЛОЖЕНИЕ Е. Руководство администратора.....87

ВВЕДЕНИЕ

Интернет-провайдер ведёт учёт абонентов, заключённых с ними договоров и сеансов связи, по которым начисляется плата. Когда провайдеров несколько и они объединены общей информационной системой, к учёту добавляются требования разграничения доступа: сотрудник провайдера должен видеть только договоры и сеансы своей фирмы, руководитель — показатели своей фирмы, а аналитический центр объединения — сводные данные по всем провайдерам. Такие требования естественно выполняются на стороне сервера баз данных: если проверка прав, ограничения целостности и расчёты находятся в базе данных, их нельзя обойти, подключившись к ней другой программой.

Целью курсового проекта является создание клиент-серверной информационной системы учёта услуг связи интернет-провайдеров (вариант Б-16) средствами СУБД PostgreSQL, в которой вся логика программного обеспечения размещена на сервере, а клиентское приложение предоставляет пользователю интерфейс в соответствии с его ролью.

Для достижения цели решаются задачи:

1) Проанализировать предметную область, построить концептуальную модель базы данных в третьей нормальной форме и обосновать допущенную денормализацию.

2) Создать базу данных средствами SQL: схемы, домены, таблицы, индексы, последовательности; заполнить справочники и основные таблицы псевдослучайными данными; секционировать таблицу сеансов связи.

3) Реализовать группы пользователей средствами PostgreSQL и ограничить видимость данных ролью и провайдером пользователя.

4) Спроектировать запросы на выборку всех основных видов, оформив запросы без параметров представлениями, а запросы с параметрами — функциями.

5) Разработать клиентское приложение на Qt 6 с постраничным выводом, поиском, составной формой «один-ко-многим», отчётами с диаграммами и выгрузкой в Excel.

6) Проверить систему автоматическими тестами, в том числе на защиту от несанкционированного доступа, одновременное изменение данных и каскадное удаление.

Пояснительная записка содержит описание предметной области, обоснование выбора СУБД и инструментальных средств, проектирование базы данных и клиентского приложения, результаты тестирования; в приложениях приведены листинги шаблонов, серверного и клиентского приложений и руководства пользователя, суперпользователя и администратора.

1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ, ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Предметная область — учёт услуг доступа в сеть Интернет, оказываемых несколькими фирмами-провайдерами. Информационная система предназначена для регионального объединения провайдеров: сотрудники каждого провайдера ведут договоры и сеансы связи своей фирмы и общий для объединения реестр абонентов, руководители провайдеров анализируют показатели своей фирмы, аналитический центр объединения получает сводные данные по всем провайдерам, администраторы ведут учётные записи пользователей, справочники и обслуживают базу данных.

В системе хранятся сведения:

– о фирмах-провайдерах: название, тип собственности (государственный, частный, ЗАО, ОАО и т. п.), город и адрес, телефон, год начала работы;

– об абонентах: ФИО или название предприятия, тип абонента (частное лицо, школа, вуз, предприятие и т. п.), город и физический адрес, телефон;

– о договорах абонентов с провайдерами: номер договора, дата подключения, стоимость подключения, стоимость 1 Мб информации, выделенный IP-адрес, адрес подключения, контактный телефон и контактное лицо, дата расторжения;

– о сеансах связи: дата и время сеанса, объём переданной информации в мегабайтах, тариф.

Один абонент может заключить несколько договоров, в том числе с разными провайдерами. Абонент хранится в реестре объединения одной записью, а сведения, которые у разных провайдеров различаются (адрес подключения, контактный телефон и лицо), — в договоре. Тариф определяет скидку к стоимости трафика в зависимости от времени сеанса: в бизнес-время оплата составляет 100 %, по вечернему тарифу предоставляется скидка 20 %, по ночному — 50 %; предусмотрены также тарифы выходного дня и тарифы,

назначаемые вручную (праздничный, образовательный, акционный, служебный). Стоимость сеанса вычисляется как произведение объёма, стоимости 1 Мб по договору и доли оплаты по тарифу.

Основные процессы предметной области: регистрация провайдера и ведение справочников (администратор приложения); регистрация абонента, заключение и расторжение договора, регистрация сеансов связи с подбором тарифа и расчётом стоимости (оператор провайдера; в реальной эксплуатации сеансы загружаются из биллинговой системы провайдера); анализ выручки, трафика и распределения сеансов по тарифам и типам абонентов за выбранные периоды (руководитель провайдера и главный аналитик объединения).

Для работы системы потребовалось уточнить ряд деталей предметной области; приняты следующие решения:

– IP-адрес хранится в договоре, а не в сведениях об абоненте: адрес назначается провайдером по конкретному договору, и абонент, заключивший несколько договоров (например, предприятие с несколькими точками подключения), имеет несколько IP-адресов;

– абонент один для всех провайдеров объединения (без дублей по наименованию и телефону), связь абонента с провайдером — договор;

– введены номер договора (уникален в пределах провайдера) и дата расторжения; для тарифа задаются временное окно и тип дня, по которым тариф назначается сеансу;

– стоимость сеанса и скидка фиксируются в момент регистрации сеанса и не меняются при последующем изменении тарифов и условий договора.

1.2 Постановка задачи

Требуется разработать клиент-серверную информационную систему на основе СУБД PostgreSQL, в которой бизнес-логика, проверка ограничений целостности, расчёты, отчёты и разграничение доступа реализованы средствами сервера базы данных, а настольное приложение на Qt выполняет функции

тонкого клиента: подключается к серверу под учётной записью пользователя, вызывает функции и представления схемы api и отображает результат. Одна программа обслуживает все роли; набор разделов и инструментов определяется группой пользователя, которую сообщает сервер после входа.

Пользователи системы и их задачи приведены в таблице 1.1.

Таблица 1.1 – Группы пользователей и их задачи

Группа	**Роль PostgreSQL**	**Задачи и видимость данных**
Оператор	grp_operator	Абоненты, договоры и сеансы связи своего провайдера; ведение договоров и абонентов, которых не обслуживают другие провайдеры
Руководитель провайдера	grp_manager	Данные и статистика своего провайдера за выбранный период; только просмотр
Главный аналитик	grp_chief	Сводная статистика по всем провайдерам с отбором по провайдеру; только просмотр
Администратор приложения	grp_admin	Учётные записи операторов, руководителей и главных аналитиков; справочники, провайдеры, сведения общих абонентов; оперативных данных не видит
Администратор БД	grp_dba	Учётные записи администраторов приложения, журнал изменений, состояние таблиц и подключений, резервное копирование; данных приложения через программу не видит

К системе предъявляются следующие требования: модель данных в третьей нормальной форме; домены, индексы, не менее шести триггеров; не менее 10 записей в каждом справочнике и 10–50 тыс. записей в основных таблицах; запросы на выборку основных видов, оформленные представлениями и функциями; модифицируемое представление; роли пользователей, защита на уровне строк и секционирование одной из основных таблиц; клиентское приложение с постраничным просмотром, поиском, составной формой «один-ко-многим», отчётами с диаграммами и выгрузкой в Excel.

2 ОБОСНОВАНИЕ ВЫБОРА СУБД, ОПИСАНИЕ ВОЗМОЖНОСТЕЙ СУБД

В качестве сервера баз данных выбрана СУБД PostgreSQL 18 — свободно распространяемая объектно-реляционная СУБД с открытым исходным кодом [3]. Выбор определён требованиями к системе (вся логика программного обеспечения — на сервере, роли пользователей — средствами СУБД, защита на уровне строк, секционирование) и возможностями, которые PostgreSQL предоставляет для их выполнения:

– роли и группы ролей: пользователь системы — роль с правом входа, член группы; права на схемы, таблицы и функции выдаются группам командами GRANT и REVOKE; пароли хранятся в системном каталоге в виде хеша SCRAM-SHA-256;

– функции на SQL и PL/pgSQL, выполняемые с правами владельца (SECURITY DEFINER) и с заданными параметрами сеанса (search_path, часовой пояс), — через них пользователи получают доступ к данным, не имея прав на таблицы;

– политики защиты на уровне строк (ROW LEVEL SECURITY), действующие и на владельца таблицы при включённом режиме FORCE;

– декларативное секционирование таблиц по диапазону, списку и хешу, в том числе многоуровневое;

– домены, проверочные ограничения, внешние ключи с выбором правила удаления, столбцы идентификаторов (GENERATED AS IDENTITY) и последовательности;

– типы данных предметной области: numeric для денежных сумм без ошибок двоичного округления, inet для IP-адресов и подсетей, timestamptz для моментов времени, jsonb для слабоструктурированных документов;

– оконные функции, группировка с ROLLUP, общие табличные выражения, частичные индексы и индексы по выражениям, пропуск по первой колонке составного индекса (Skip Scan, появился в версии 18);

– инструменты анализа плана выполнения запросов (EXPLAIN ANALYZE, модуль auto_explain) и сбора статистики.

Сервер установлен на компьютере разработчика под управлением Windows 11 (служба postgresql-x64-18, порт 5432). Параметры, влияющие на использование памяти и ресурсов, изменены командой ALTER SYSTEM (запись в файл postgresql.auto.conf) с последующим перезапуском службы; значения приведены в таблице 2.1.

Таблица 2.1 – Параметры сервера PostgreSQL

Параметр	**Значение**	**Обоснование**
shared_buffers	512MB (было 128MB)	Для Windows рекомендуется 64–512 МБ с опорой на кэш операционной системы; компьютер не является выделенным сервером
work_mem	64MB (было 4MB)	Свободная память около 10 ГБ, до 35 одновременных запросов по две операции сортировки или хеширования: (10 ГБ / 2 − 0,5 ГБ) / 70 ≈ 66 МБ
temp_buffers	32MB (было 8MB)	Верхняя граница рекомендуемого диапазона 16–32 МБ для временных таблиц
max_prepared_transactions	5 (было 0)	Возможность двухфазной фиксации транзакций; требует перезапуска сервера
max_files_per_process	1000	Значение по умолчанию задано явно; ошибок нехватки дескрипторов файлов нет
track_counts	on	Сбор статистики обращений к таблицам, необходимый для автоочистки и представлений pg_stat_*

Так как функции смены и сброса пароля передают новый пароль серверу параметром запроса, для журнала сервера установлено log_parameter_max_length = 0: значения параметров не записываются в журнал ни для медленных, ни для ошибочных запросов. Пароли администраторов базы данных меняются командой \password программы psql, которая отправляет на сервер уже вычисленный хеш.

Кроме реляционных таблиц PostgreSQL хранит слабоструктурированные данные в типе jsonb — двоичном представлении документа JSON с индексируемым доступом к ключам. В предметной области такие данные — учётные записи сеансов, которые биллинг провайдера выгружает по каждому сеансу: набор атрибутов зависит от производителя оборудования доступа и меняется со сменой оборудования, поэтому запись хранится целиком, а нужные поля извлекаются запросами. Для работы с документами СУБД предоставляет операторы извлечения значения по ключу и пути, проверки вхождения образца и наличия ключей, язык путей SQL/JSON (jsonpath), функции разбора документа в строки таблицы и сборки документа из строк, изменения части документа. Поиск ускоряют обратные индексы GIN по документу целиком и индексы B-дерева по выражению над отдельным ключом. На 100 025 учётных записях, построенных из данных системы в лабораторной работе № 7, индексы сократили время типовых запросов с десятков миллисекунд до долей миллисекунды. В самой системе тип jsonb предназначен для журнала изменений audit.change_log: строка до и после изменения хранится документом, поэтому журнал не зависит от структуры таблиц.

Секционирование позволяет хранить большую таблицу частями (секциями) по значению ключа: запросы за период читают только нужные секции, а устаревшие данные отсоединяются целиком. Для системы кандидатом является таблица сеансов связи — единственная неограниченно растущая таблица; её первичный ключ уже включает дату сеанса, как того требует секционирование по дате. Применение секционирования описывается в подразделе 4.6.

Для администрирования используется pgAdmin 4 (просмотр объектов, построение схемы данных, выполнение запросов) и программа psql (сценарии создания и пересборки базы, автоматические тесты).

3 ОБОСНОВАНИЕ ВЫБОРА ИНСТРУМЕНТАЛЬНЫХ СРЕДСТВ ДЛЯ НАПИСАНИЯ КЛИЕНТСКОЙ ЧАСТИ, ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПО

Клиентское приложение разработано на языке C++17 с библиотекой Qt 6 (модули Widgets, Sql, Charts, PrintSupport, Svg) и системой сборки CMake [4]. Выбор обусловлен следующим:

– модуль Qt Sql содержит драйвер QPSQL, работающий через клиентскую библиотеку PostgreSQL libpq, и позволяет выполнять подготовленные запросы с привязкой параметров (prepare и bindValue), что исключает внедрение SQL-кода через пользовательский ввод;

– архитектура «модель — представление» Qt (QAbstractTableModel, QTableView) отделяет загрузку данных от их отображения, что позволяет вынести постраничную загрузку из функций сервера в один общий класс модели;

– Qt Charts строит диаграммы отчётов, модуль PrintSupport печатает их, а библиотека QXlsx сохраняет отчёты в формате Excel без установленного Microsoft Office;

– формы проектируются в Qt Designer, единый стиль экранов задаётся таблицей стилей Qt (QSS), а программа компилируется в исполняемый файл Windows, которому нужны только библиотеки Qt и libpq.

Приложение — тонкий клиент: оно подключается к PostgreSQL под учётной записью пользователя, вызывает функции и представления схемы api и отображает результат. SQL-текст в клиенте формируется только вида «SELECT * FROM api.<функция>(<параметр> => :<параметр>, …)» с проверкой имени объекта; значения передаются параметрами. Пароль не сохраняется: в файле config.ini рядом с программой хранятся только адрес сервера, порт, имя базы и последний логин. Структура исходного кода приведена в таблице 3.1.

Таблица 3.1 – Структура исходного кода клиента

Каталог	Содержимое
src/app	Параметры подключения (AppConfig), состав разделов и навигации по группе пользователя (RoleConfig), ключи отладочной сборки (DevOptions)
src/db	Подключение к серверу (Db) и вызов функций и представлений api (ApiCall)
src/models	Модель постраничного списка PagedTableModel
src/ui/widgets	Общие визуальные компоненты: таблица со страницами DataGrid, панель страниц PagerBar, элементы форм FormKit
src/ui/pages	Страницы разделов: начальная, абоненты, договоры, сеансы связи, провайдеры, статистика, пользователи, страницы администратора БД
src/ui/dialogs	Карточки договора и абонента, формы выбора, создание пользователя, смена и сброс пароля
src/charts, src/export	Диаграммы отчётов (ReportChart) и выгрузка в Excel (ExcelExport)

3.1 Невизуальные компоненты для работы с данными

Класс Db создаёт единственное подключение QPSQL под логином и паролем пользователя, после подключения вызывает функцию api.whoami и заполняет структуру Whoami (логин, ФИО, группа, название роли, провайдер, версия сервера); по ней строится весь интерфейс. Метод Db::userMessage извлекает из ошибки сервера текст сообщения RAISE EXCEPTION без служебных префиксов драйвера — пользователь видит, например, «Недостаточно прав для этого действия».

Класс ApiCall — единственный путь клиента к данным. Метод select формирует вызов функции api с именованными параметрами, проверяя, что имена функции и параметров состоят только из строчных латинских букв, цифр и знака подчёркивания, и передаёт значения через bindValue (рисунок 3.1). Методы selectView и selectViewRow читают страницу представления и одну строку представления по коду.

```
bool ApiCall::select(const QString &function, const QList<ApiParam> &params, QSqlQuery *query, QString *error)
{
    QStringList names{function};
    QStringList args;
    for (const ApiParam &p : params) {
        names << p.name;
        args << QStringLiteral("%1 => :%1").arg(p.name);
    }
    if (!checkIdentifiers(names, error))
        return false;
    return run(QStringLiteral("SELECT * FROM api.%1(%2)").arg(function, args.join(QStringLiteral(", "))),
               params, query, error);
}
```

Рисунок 3.1 – Вызов функции схемы api с именованными параметрами (ApiCall::select)

Модель PagedTableModel загружает страницу списка: к параметрам отбора добавляются p_limit и p_offset, общее число строк берётся из колонки total_count первой строки результата, итоговая строка «Итого…» формируется из полей, посчитанных сервером оконными функциями по всему отбору (рисунок 3.2). Код строки (id) в колонки не выводится, а возвращается моделью в роли Qt::UserRole — идентификаторы записей пользователю не показываются.

```
bool PagedTableModel::reload()
{
    QSqlQuery q;
    bool ok;
    if (m_isView) {
        ok = ApiCall::selectView(m_source, m_orderBy, m_limit, m_offset, &q, &m_error);
    } else {
        QList<ApiParam> params = m_filter;
        params << ApiParam{QStringLiteral("p_limit"), m_limit} << ApiParam{QStringLiteral("p_offset"), m_offset};
        ok = ApiCall::select(m_source, params, &q, &m_error);
    …
}
```

Рисунок 3.2 – Загрузка страницы списка (PagedTableModel::reload, начало)

Класс AppConfig хранит параметры подключения в config.ini, RoleConfig по группе возвращает разделы главного окна, ExcelExport сохраняет отчёт в файл .xlsx.

3.2 Визуальные компоненты отображения данных

Окно входа «Доступ к информационной базе» содержит поля пользователя и пароля и раскрывающийся блок параметров подключения (рисунок 3.3).

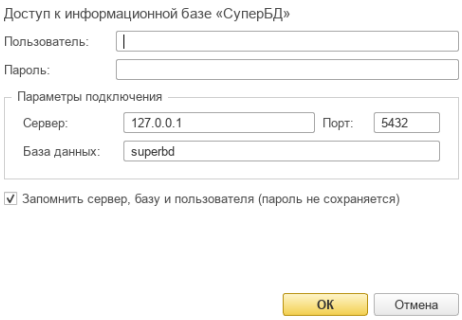


Рисунок 3.3 – Окно входа в программу

Главное окно (рисунок 3.4) состоит из панели разделов слева (набор разделов зависит от группы пользователя), панели навигации раздела, вкладок открытых форм и строки состояния. Начальная страница показывает сведения о пользователе и блок «Текущие дела» — счётчики, открывающие список или отчёт.

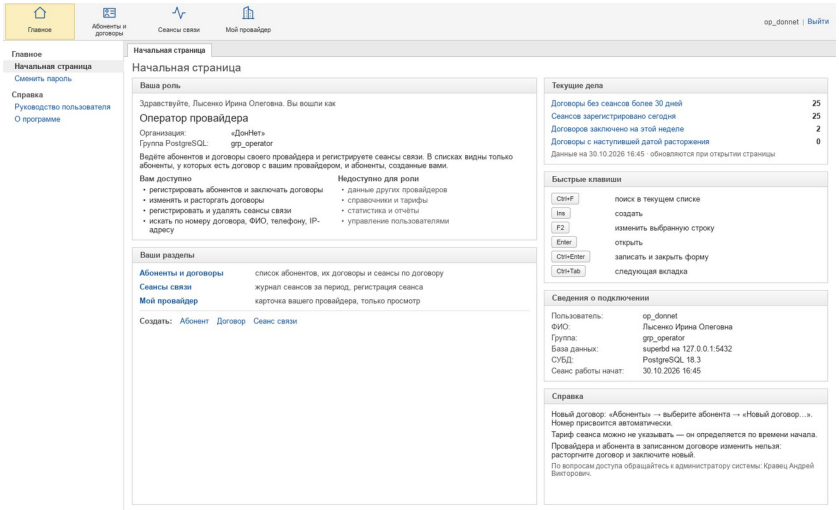


Рисунок 3.4 – Главное окно программы (оператор, начальная страница)

Общие визуальные компоненты: DataGrid — таблица (QTableView) с моделью PagedTableModel и панелью страниц PagerBar («Показывать по», «Страница N из M»), сообщающая о выборе строки и двойном щелчке; FormKit — заголовок формы, командная панель с полем поиска, поле с кнопкой выбора «…»; PickerDialog — форма выбора записи из постраничного списка; ReportChart — диаграмма отчёта на основе Qt Charts (накопительные столбцы, линии, горизонтальные столбцы) с легендой.

3.3 Разработка шаблонов приложений для работы с таблицами базы данных

Для работы с таблицами базы данных разработаны три шаблона страниц (исходный код — в приложении А).

Шаблон списка: заголовок формы, командная панель, строка параметров отбора, таблица DataGrid, связанная с функцией api, и подсказка. Параметры отбора передаются модели как параметры функции, после изменения отбора загружается первая страница. По этому шаблону построены страницы «Договоры», «Сеансы связи», «Провайдеры», «Пользователи», «Общие абоненты», «Договоры без сеансов» и страницы администратора БД. Построение страницы по шаблону показано на рисунке 3.5.

```
IdleContractsPage::IdleContractsPage(const Whoami &me, QWidget *parent)
    : QWidget(parent)
{
    Q_UNUSED(me);
    QVBoxLayout *layout = FormKit::formLayout(this, QStringLiteral("Договоры без сеансов"));

    FormKit::CommandBar bar(this);
    QPushButton *open = bar.button(QStringLiteral("Открыть договор"), true);
    QPushButton *refresh = bar.button(QStringLiteral("Обновить"));
    bar.stretch();
    bar.more();
    layout->addLayout(bar.layout());
…
}
```

Рисунок 3.5 – Страница списка по шаблону (IdleContractsPage)

Составная форма «один-ко-многим» содержит несколько DataGrid, связанных выбором строки: при выборе абонента загружаются его договоры, при выборе договора — сеансы договора с итогами. По этому шаблону построена страница «Абоненты» (подраздел 5.1).

Шаблон отчёта: строка параметров (период, периодичность, провайдер, тип абонента), табличный документ с итоговой строкой и диаграмма ReportChart, команды «Сохранить в Excel…» и «Печать». Данные отчёта целиком формирует функция api.report_*, клиент только раскладывает их по колонкам и рядам диаграммы. По этому шаблону построены отчёты раздела «Статистика» (подраздел 5.6).

4 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ В ВЫБРАННОЙ СУБД

4.1 Проектирование концептуальной модели БД

Модель построена по методике IDEF1X [1]. Исходное отношение, содержащее все сведения предметной области в одной таблице сеансов, приводит к аномалиям обновления (смена телефона провайдера требует изменения всех строк его сеансов), включения (невозможно зарегистрировать провайдера без сеансов) и удаления (удаление последнего сеанса уничтожает сведения о договоре) [2]. Последовательное приведение к первой нормальной форме (атомарность значений, развёртывание повторяющейся группы сеансов), второй (устранение зависимостей реквизитов провайдера и договора от части составного ключа) и третьей (вынесение абонента и тарифа, от которых транзитивно зависят реквизиты абонента и скидка) даёт восемь сущностей предметной области. Концептуальная модель показана на рисунке 4.1.

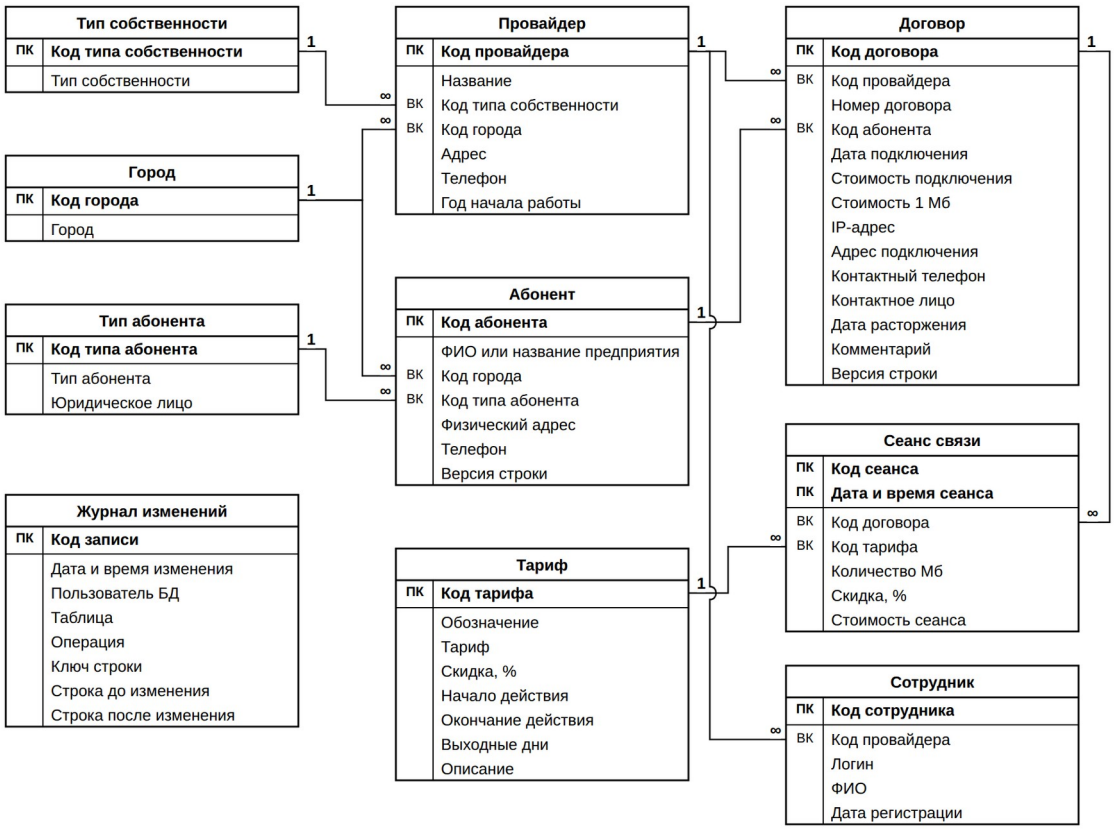


Рисунок 4.1 – Концептуальная модель базы данных

Справочники: тип собственности, город, тип абонента, тариф. Базовые таблицы: провайдер, абонент, договор, сеанс связи. Абоненты образуют общий реестр объединения: одно лицо или предприятие, обслуживаемое несколькими провайдерами, представлено одной записью, а дубли исключает уникальность наименования (без учёта регистра) вместе с цифрами телефона. Провайдер и абонент связаны «многие ко многим» через договор — таблицу пересечения, в которой, кроме номера, цен и IP-адреса, хранятся сведения, различающиеся у разных провайдеров: адрес подключения, контактный телефон и контактное лицо. Для требований к системе добавлены две служебные таблицы: «Сотрудник» связывает учётную запись PostgreSQL с провайдером для разграничения доступа к строкам (паролей и ролей таблица не содержит — роль определяется членством в группе PostgreSQL), «Журнал изменений» предназначен для версий изменённых строк в формате jsonb и внешними ключами не связан, чтобы записи сохранялись после удаления строк. Пользователи и группы хранятся в системных таблицах pg_authid и pg_auth_members.

Первичные ключи всех таблиц — суррогатные целочисленные, генерируемые СУБД. Естественные ключи сохранены как альтернативные: название провайдера и наименования в справочниках уникальны, номер договора уникален в пределах провайдера, IP-адрес уникален среди действующих договоров провайдера (частичный уникальный индекс). Первичные ключи двух секционированных таблиц (подраздел 4.6) составные, потому что PostgreSQL требует включения ключа секционирования в первичный ключ: у сеанса — идентификатор и дата сеанса, у договора — идентификатор и код провайдера. Поэтому сеанс ссылается на договор составным внешним ключом «код договора, код провайдера» и хранит код провайдера договора; провайдер договора не меняется, а согласованность значений обеспечивает внешний ключ.

Модель содержит обоснованную денормализацию: в сеансе хранятся скидка тарифа и стоимость сеанса на момент его регистрации. Скидка транзитивно зависит от тарифа, а стоимость вычисляется из объёма, цены договора и скидки, однако изменение справочника тарифов или условий договора не должно менять стоимость уже оказанных услуг, а отчёты о выручке по сотням тысяч сеансов не должны пересчитывать стоимость с соединением трёх таблиц. Значения задаются при вставке сеанса и не редактируются, поэтому аномалии обновления не возникают; соответствие скидки тарифу и стоимости формуле проверяется автоматическим тестом 3.4.

4.2 Создание таблиц, доменов, индексов, сиквенсов

База данных superbd создана в кодировке UTF-8 и разделена на схемы по назначению: ref — справочники и домены, core — основные таблицы предметной области, sec — сотрудники и функции разграничения доступа, audit — журнал изменений, api — функции и представления, через которые работает клиентское приложение. Все объекты принадлежат роли app_owner, у которой нет права входа в систему; права на схемы у роли PUBLIC отозваны, группам пользователей выдаётся доступ только к схеме api.

Структура создаётся последовательностью сценариев миграции (приложение Б), которые выполняются по порядку из сценария db/build.sql: схемы и группы пользователей, домены, справочники, основные таблицы, триггер проверки логина сотрудника, индексы, функции разграничения доступа, представления и функции api, после чего таблицы заполняются данными. Таблицы создаются в порядке «сначала сторона «один», затем подчинённые»; каждый сценарий можно выполнить повторно. Создание основной таблицы «Договор» показано на рисунке 4.2.

```
CREATE TABLE IF NOT EXISTS core.contract (
    id              integer          GENERATED ALWAYS AS IDENTITY,
    provider_id     integer          NOT NULL,
    contract_no     ref.d_contract_no NOT NULL
        DEFAULT ('Д-' || to_char(current_date, 'YYYY') || '-' || lpad(nextval('core.contract_no_seq')::text, 6, '0')),
    subscriber_id   integer          NOT NULL,
    connected_on    date             NOT NULL DEFAULT current_date,
    connection_fee  ref.d_money      NOT NULL,
    price_per_mb    ref.d_price_mb   NOT NULL,
    ip_address      inet             NOT NULL,
    connection_address ref.d_address NOT NULL,
    contact_phone   ref.d_phone,
    contact_person  ref.d_name,
    closed_on       date,
    note            text,
    row_version     integer          NOT NULL DEFAULT 1,
    CONSTRAINT pk_contract PRIMARY KEY (id, provider_id),
    CONSTRAINT uq_contract_provider_id_contract_no UNIQUE (provider_id, contract_no),
    CONSTRAINT fk_contract_provider_id FOREIGN KEY (provider_id)
        REFERENCES core.provider (id) ON DELETE RESTRICT,
    CONSTRAINT fk_contract_subscriber_id FOREIGN KEY (subscriber_id)
        REFERENCES core.subscriber (id) ON DELETE CASCADE,
    CONSTRAINT ck_contract_closed_after_connected CHECK (closed_on >= connected_on),
    CONSTRAINT ck_contract_row_version CHECK (row_version > 0)
) PARTITION BY LIST (provider_id);
```

Рисунок 4.2 – Создание таблицы «Договор»

Для каждой таблицы и колонки задан комментарий (COMMENT ON) с названием из концептуальной модели. Ограничения значений заданы доменами и проверочными ограничениями: дата расторжения договора не раньше даты подключения, время начала и окончания действия тарифа задаются вместе. Допустимые символы в доменах проверяются функцией translate, без регулярных выражений. Связи между таблицами реализованы внешними ключами с явно выбранным правилом удаления: удаление абонента каскадно удаляет его договоры, удаление договора — его сеансы, а удаление провайдера, тарифа или записи справочника, на которые есть ссылки, запрещено. Внешних ключей на системные таблицы PostgreSQL поставить нельзя, поэтому соответствие логина сотрудника существующей учётной записи PostgreSQL проверяет триггер (подраздел 4.3).

Кроме индексов, которые PostgreSQL создаёт для первичных ключей и ограничений уникальности, проиндексированы все внешние ключи, дата и время сеанса и пара «договор, дата сеанса» для выборок за период, пара «провайдер, дата подключения» договора и пара «абонент, провайдер» договора, по которой

проверяется, есть ли у провайдера договор с абонентом. Уникальный индекс по выражениям «наименование в нижнем регистре с классом операторов text_pattern_ops, цифры телефона» не допускает повторной записи абонента и одновременно служит для поиска абонента по началу наименования. Уникальность IP-адреса среди действующих договоров провайдера обеспечивает частичный уникальный индекс с условием closed_on IS NULL. Номер договора по умолчанию формируется из явной последовательности core.contract_no_seq в виде «Д-<год>-<шестизначный номер>»; остальные десять последовательностей созданы СУБД для колонок идентификаторов (GENERATED ALWAYS AS IDENTITY) основных таблиц, справочников, сотрудников и журнала изменений.

Таблицы заполнены сценариями db/seed: справочники — осмысленными значениями (от 10 до 12 записей), основные таблицы — псевдослучайными данными, сгенерированными средствами SQL (generate_series, random с фиксированным начальным значением setseed, поэтому результат воспроизводим): 30 провайдеров, 8 000 абонентов, 12 000 договоров и 500 000 сеансов связи — всего более 520 000 строк; сеансы распределены по договорам пропорционально сроку их действия. Сценарий seed/040 добавляет провайдера без договоров и сотрудников, два договора и сеансы за текущий день — без них часть запросов и счётчиков начальной страницы возвращала бы пустой результат. Генератор соблюдает ограничения предметной области: сеансы попадают в период действия договора, тариф выбирается по времени сеанса и типу дня, стоимость сеанса вычисляется по формуле; 5 % абонентов не имеют договоров, а два тарифа не имеют сеансов — это нужно для демонстрации внешних соединений.

4.3 Разработка триггеров

Триггер — процедура, которую сервер вызывает автоматически при вставке, изменении или удалении строк таблицы. В системе реализован триггер

trg_app_user_biu_check_role таблицы «Сотрудник» (рисунок 4.3): перед вставкой строки и перед изменением логина он проверяет, что логин является существующей учётной записью PostgreSQL с правом входа. Триггер заменяет внешний ключ на системный каталог pg_roles, который поставить нельзя. Проверка выполняется автоматическим тестом 4.4: сотрудник с несуществующим логином отклоняется.

```
CREATE OR REPLACE FUNCTION sec.trg_app_user_biu_check_role()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = pg_catalog, pg_temp
AS $$
BEGIN
   IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_roles r
                  WHERE r.rolname = NEW.login AND r.rolcanlogin) THEN
      RAISE EXCEPTION 'Учётная запись «%» не найдена: сначала создайте пользователя PostgreSQL с правом входа',
NEW.login
         USING ERRCODE = 'P0001';
   END IF;
   RETURN NEW;
END $$;
COMMENT ON FUNCTION sec.trg_app_user_biu_check_role() IS
   'Триггерная функция: логин сотрудника должен существовать в pg_roles и иметь право входа (связь R10)';

DROP TRIGGER IF EXISTS trg_app_user_biu_check_role ON sec.app_user;
CREATE TRIGGER trg_app_user_biu_check_role
   BEFORE INSERT OR UPDATE OF login ON sec.app_user
   FOR EACH ROW EXECUTE FUNCTION sec.trg_app_user_biu_check_role();
COMMENT ON TRIGGER trg_app_user_biu_check_role ON sec.app_user IS
   'Проверка, что логин сотрудника — существующая LOGIN-роль PostgreSQL';
```

Рисунок 4.3 – Триггерная функция и триггер проверки логина сотрудника

[ЗАГЛУШКА: остальные триггеры (подбор тарифа и расчёт стоимости сеанса, неизменность сторон договора, запрет удаления провайдера, журнал изменений, INSTEAD OF) будут добавлены после реализации]

4.4 Организация многоролевого доступа к данным

Роли пользователей реализованы средствами PostgreSQL. Пользователь системы — роль с правом входа (LOGIN), член ровно одной из пяти групп (таблица 4.1); пароль хранит только СУБД. Таблицы пользователей с паролями или ролями в базе нет: группа пользователя определяется по системному каталогу pg_auth_members, а таблица «Сотрудник» хранит только ФИО и привязку к провайдеру.

Таблица 4.1 – Группы пользователей и их права

Группа	Права
grp_operator	Функции и представления api своих экранов: абоненты, договоры, сеансы своего провайдера, изменение договоров и своих абонентов
grp_manager	Те же данные своего провайдера только для чтения, отчёты «Статистики»
grp_chief	Данные и отчёты по всем провайдерам или выбранному, только чтение
grp_admin	Пользователи операторов, руководителей и главных аналитиков, провайдеры, справочники, общие абоненты
grp_dba	Пользователи-администраторы приложения, журнал изменений, таблицы, подключения; встроенные роли pg_monitor, pg_maintain и pg_read_all_data (чтение для резервного копирования); переключение на владельца командой SET ROLE app_owner

Права выдаются по принципу минимальных привилегий:

– у роли PUBLIC отозваны права на базу данных и все схемы проекта, а командой ALTER DEFAULT PRIVILEGES отозвано право выполнения любых новых функций владельца;

– группы получают право подключения к базе, USAGE на схему api и EXECUTE на функции (SELECT на представления) только своих экранов; прав на таблицы ни у одной группы нет, кроме чтения у администратора БД;

– функции api принадлежат app_owner и выполняются с его правами (SECURITY DEFINER) с фиксированным путём поиска search_path = pg_catalog, pg_temp, исключающим подмену объектов, и часовым поясом предметной области;

– первая операция каждой функции api — проверка группы вызывающего функцией sec.require_group; отказ возвращается ошибкой 42501 «Недостаточно прав для этого действия», а пока пользователь не сменил временный пароль — ошибкой SB001.

Группа определяется по прямому членству session_user в группах проекта (рисунок 4.4). Встроенная функция pg_has_role для этого не подходит: она учитывает и косвенное членство, а администратор БД через членство в app_owner формально состоял бы во всех группах.

```
CREATE OR REPLACE FUNCTION sec.group_of(p_login name)
RETURNS text
LANGUAGE sql STABLE
SET search_path = pg_catalog, pg_temp
AS $$
    SELECT g.rolname::text
    FROM pg_catalog.pg_auth_members m
    JOIN pg_catalog.pg_roles g ON g.oid = m.roleid
    JOIN pg_catalog.pg_roles u ON u.oid = m.member
    WHERE u.rolname = p_login
      AND m.inherit_option
      AND g.rolname IN ('grp_admin', 'grp_dba', 'grp_operator', 'grp_manager', 'grp_chief')
    ORDER BY array_position(ARRAY['grp_admin', 'grp_dba', 'grp_operator', 'grp_manager', 'grp_chief'], g.rolname::text)
    LIMIT 1
$$;
```

Рисунок 4.4 – Определение группы пользователя (sec.group_of)

Учётные записи создаются из приложения функцией api.user_create, принадлежащей app_owner с правом CREATEROLE: функция проверяет, что вызывающий администратор управляет указанной группой, выполняет CREATE ROLE и GRANT, добавляет строку «Сотрудник» и выдаёт временный пароль на сутки. Администратор приложения создаёт операторов, руководителей и главных аналитиков, администратор БД — администраторов приложения; первого администратора БД создаёт суперпользователь postgres. Вход пользователю запрещается и разрешается функцией api.user_set_login, пароль сбрасывается функцией api.user_reset_password: сервер генерирует временный пароль и до его смены пользователю доступны только сведения о себе и смена пароля. Группы и пользователи в pgAdmin показаны на рисунке 4.5.

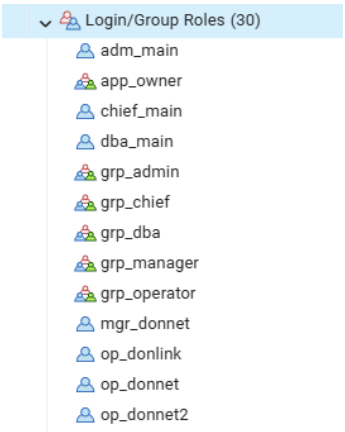


Рисунок 4.5 – Группы и учётные записи пользователей в pgAdmin

4.5 Разграничение доступа к данным на уровне строк (в зависимости от роли и логина)

Оператор и руководитель видят абонентов, договоры и сеансы только своего провайдера, главный аналитик — всех провайдеров или выбранного, администраторам данные абонентов, договоров и сеансов не выдаются. Провайдер пользователя определяется по его логину (session_user) в таблице «Сотрудник», группа — по членству в группах PostgreSQL.

Ограничение строк реализовано в функциях и представлениях api. Функции sec.scope_data и sec.scope_providers возвращают массив кодов провайдеров, данные которых видит пользователь, с учётом отбора по провайдеру из формы: свой провайдер, все провайдеры или пустой массив (рисунок 4.6). Каждый запрос к договорам и сеансам содержит одно условие «provider_id = ANY (массив)», вычисляемое один раз до выполнения запроса, что позволяет использовать индекс по провайдеру. Абонент общего реестра виден оператору и руководителю, пока у их провайдера есть с ним договор. Чужой договор не возвращается ни в списке, ни в карточке; договор с чужим абонентом по коду заключить нельзя. Эти правила проверены автоматическими тестами 7.x и 9.x (раздел 6).

```
CREATE OR REPLACE FUNCTION sec.provider_scope(p_sees_all boolean, p_provider_id integer)
RETURNS integer[]
LANGUAGE plpgsql STABLE
SET search_path = pg_catalog, pg_temp
AS $$
DECLARE
   v_own integer;
BEGIN
   PERFORM sec.assert_password_changed();
   IF p_sees_all THEN
      IF p_provider_id IS NOT NULL THEN
         RETURN ARRAY[p_provider_id];
      END IF;
      RETURN ARRAY(SELECT p.id FROM core.provider p ORDER BY p.id);
   END IF;
   v_own := sec.current_provider_id();
   IF v_own IS NULL OR v_own <> coalesce(p_provider_id, v_own) THEN
      RETURN '{}';
   END IF;
   RETURN ARRAY[v_own];
END $$;
```

Рисунок 4.6 – Видимые пользователю провайдеры (sec.provider_scope)

[ЗАГЛУШКА: политики защиты на уровне строк для договоров, сеансов и абонентов будут добавлены после реализации]

4.6 Партицирование одной из основных таблиц БД

Секционированы две основные таблицы. Таблица сеансов связи — единственная неограниченно растущая таблица системы со сроком хранения сведений; она секционирована по диапазону даты и времени начала сеанса в два уровня: секции годов разделены на вложенные квартальные секции, а сеансы года, секция которого ещё не создана, попадают в секцию по умолчанию. Год служит единицей архивирования: устаревший год отсоединяется (DETACH PARTITION) и удаляется без удаления отдельных строк. Квартал служит единицей обслуживания и отсечения: очистка и сбор статистики выполняются по кварталу, а отчёты и журналы за период читают только свои кварталы. Таблица договоров секционирована по списку провайдеров: у каждого провайдера своя секция, сотрудники провайдера работают только с ней, аналитические отчёты — с таблицей целиком. Создание таблицы сеансов показано на рисунке 4.7, объявление секционирования договоров — в последней строке рисунка 4.2.

```
CREATE TABLE IF NOT EXISTS core.session (
    id          bigint       GENERATED ALWAYS AS IDENTITY,
    started_at  timestamptz  NOT NULL,
    contract_id integer      NOT NULL,
    provider_id integer      NOT NULL,
    …
    CONSTRAINT pk_session PRIMARY KEY (id, started_at),
    CONSTRAINT fk_session_contract_id_provider_id FOREIGN KEY (contract_id, provider_id)
        REFERENCES core.contract (id, provider_id) ON DELETE CASCADE,
    CONSTRAINT fk_session_tariff_id FOREIGN KEY (tariff_id)
        REFERENCES ref.tariff (id) ON DELETE RESTRICT
) PARTITION BY RANGE (started_at);
```

Рисунок 4.7 – Создание секционированной таблицы «Сеанс связи» (фрагмент)

Абоненты по провайдерам не секционированы. Реестр абонентов общий для всех провайдеров, и 590 из 8 000 абонентов обслуживаются несколькими провайдерами; уникальность записи абонента (наименование и цифры телефона) на секционированной таблице задать нельзя, так как уникальный индекс обязан

включать ключ секционирования. Сведения, различающиеся у провайдеров, хранятся в договоре — именно он разделён по провайдерам.

Секции создаются функциями, а не сценарием с перечнем границ. Функция sec.session_partitions_ensure для каждого года периода создаёт таблицу года, присоединяет её к таблице сеансов, создаёт внутри неё четыре квартальные секции и переносит в них сеансы этого года из секции по умолчанию (рисунок 4.8). Присоединение готовой таблицы (ATTACH PARTITION) блокирует таблицу сеансов слабее, чем создание секции командой CREATE TABLE … PARTITION OF, поэтому регистрация сеансов при этом не останавливается. При сборке базы создаются секции 2024–2027 годов, затем администратор базы данных раз в год создаёт секции следующего года функцией api.db_session_partitions_ensure. Секцию договоров нового провайдера создаёт триггер после вставки строки провайдера. Индексы объявлены на секционированных таблицах, и PostgreSQL строит их в каждой секции.

```
FOR v_year IN extract(year FROM p_from)::integer .. extract(year FROM p_to)::integer LOOP
   v_name := 'session_' || v_year;
   CONTINUE WHEN to_regclass('core.' || v_name) IS NOT NULL;
   v_from := make_date(v_year, 1, 1);
   v_to := make_date(v_year + 1, 1, 1);
   SELECT array_agg(d) INTO v_rows
   FROM core.session_default d
   WHERE d.started_at >= v_from::timestamptz AND d.started_at < v_to::timestamptz;
   DELETE FROM core.session_default d
   WHERE d.started_at >= v_from::timestamptz AND d.started_at < v_to::timestamptz;
   EXECUTE format('CREATE TABLE core.%I (LIKE core.session INCLUDING DEFAULTS INCLUDING CONSTRAINTS) '
            'PARTITION BY RANGE (started_at)', v_name);
   EXECUTE format('ALTER TABLE core.session ATTACH PARTITION core.%I FOR VALUES FROM (%L) TO (%L)',
            v_name, v_from::timestamptz, v_to::timestamptz);
   …
   FOR v_q IN 1 .. 4 LOOP
      EXECUTE format('CREATE TABLE core.%I PARTITION OF core.%I FOR VALUES FROM (%L) TO (%L)',
               v_name || '_q' || v_q, v_name,
               make_date(v_year, 3 * v_q - 2, 1)::timestamptz,
               (make_date(v_year, 3 * v_q - 2, 1) + interval '3 months')::timestamptz);
   …
   END LOOP;
   INSERT INTO core.session OVERRIDING SYSTEM VALUE
   SELECT (r).* FROM unnest(v_rows) AS r;
```

Рисунок 4.8 – Создание секций года и кварталов (фрагмент функции sec.session_partitions_ensure)

В секциях 2024, 2025 и 2026 годов 64 540, 190 887 и 244 598 сеансов, секция 2027 года и секции по умолчанию пусты; в секциях провайдеров от 361 до 449 договоров. Дерево секций в pgAdmin показано на рисунках 4.9 и 4.10.

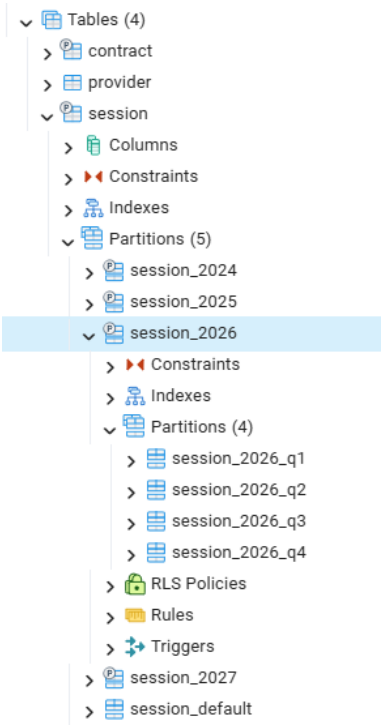


Рисунок 4.9 – Секции таблицы сеансов в pgAdmin



Рисунок 4.10 – Секции таблицы договоров в pgAdmin

Условия на дату сеанса и на провайдера позволяют планировщику отсечь лишние секции: в плане журнала сеансов оператора за месяц читаются одна квартальная секция сеансов из 17 и одна секция договоров из 32 (рисунок 4.11). Если граница периода передаётся параметром функции, лишние секции отбрасываются при выполнении запроса (в плане — строка Subplans Removed).

```
Aggregate  (cost=1186.84..1186.85 rows=1 width=40) (actual time=1.847..1.847 rows=1.00 loops=1)
  Buffers: shared hit=337
  ->  Hash Join  (cost=17.52..1186.64 rows=38 width=6) (actual time=0.058..1.795 rows=965.00 loops=1)
        Hash Cond: (s.contract_id = c.id)
        Buffers: shared hit=337
        ->  Index Scan using session_2026_q3_started_at_idx on session_2026_q3 s  (cost=0.29..1166.92 rows=953 width=14) (actual
time=0.008..1.660 rows=965.00 loops=1)
              …
              Filter: (provider_id = 1)
              Rows Removed by Filter: 26050
              Index Searches: 1
              Buffers: shared hit=330
        ->  Hash  (cost=12.11..12.11 rows=409 width=8) (actual time=0.049..0.049 rows=409.00 loops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 24kB
              Buffers: shared hit=7
              ->  Seq Scan on contract_p1 c  (cost=0.00..12.11 rows=409 width=8) (actual time=0.005..0.031 rows=409.00 loops=1)
                    Filter: (provider_id = 1)
                    Buffers: shared hit=7
```

Рисунок 4.11 – План журнала сеансов оператора за сентябрь 2026 года

Для сравнения созданы обычные таблицы с теми же данными и индексами. Время выполнения запросов системы на секционированных и обычных таблицах приведено в таблице 4.2.

Таблица 4.2 – Сравнение секционированных и обычных таблиц

Запрос	**Читаются секции**	**С секциями, мс**	**Без секций, мс**
Журнал сеансов оператора за месяц	1 квартал, 1 провайдер	3,4	2,9
Выручка по провайдерам за квартал	1 квартал, все провайдеры	32,6	30,6
Выручка за всё время	все	241	247
Сеансы договора без периода	все кварталы	0,15	0,02
Договоры провайдера за год	1 провайдер	0,036	0,043
Договоры без сеансов более 30 дней (запрос 14)	все кварталы	58,6	3,4

На 500 000 сеансов секционирование почти не ускоряет запросы, для которых есть подходящий индекс, а запросы без условия на ключ секционирования выполняются медленнее: каждая секция просматривается отдельно. Выигрыш секционирования — в обслуживании и жизненном цикле данных: отсоединение и удаление секции 2024 года (64 540 сеансов) заняли 16 и 7 мс, тогда как удаление тех же строк из обычной таблицы — 21 мс и оставило мёртвые версии строк до очистки. Цена секций договоров — объём: 32 секции со своими индексами занимают 6,7 МБ против 3,3 МБ у одной таблицы.

4.7 Проектирование запросов к базе данных

4.7.1 Симметричное внутреннее соединение с условием

Запрос 1 выбирает сотрудников провайдера по коду провайдера (условие по внешнему ключу); группа и признак входа берутся из системных каталогов pg_roles и pg_auth_members. Используется на экранах «Провайдеры» и «Мой провайдер» (рисунки 4.12 и 4.13).

```
SELECT u.id, u.login::text, u.full_name::text, g.rolname::text,
       CASE g.rolname WHEN 'grp_admin' THEN 'Администраторы' WHEN 'grp_operator' THEN 'Операторы'
                      WHEN 'grp_manager' THEN 'Руководители' WHEN 'grp_chief' THEN 'Главные аналитики' END,
       r.rolcanlogin, r.rolvaliduntil,
       count(*) OVER ()
FROM sec.app_user u
JOIN pg_catalog.pg_roles r ON r.rolname = u.login
JOIN pg_catalog.pg_auth_members m ON m.member = r.oid
JOIN pg_catalog.pg_roles g ON g.oid = m.roleid
                          AND g.rolname IN ('grp_admin', 'grp_operator', 'grp_manager', 'grp_chief')
WHERE u.provider_id = ANY (v_scope)
ORDER BY array_position(ARRAY['grp_manager', 'grp_operator'], g.rolname::text), u.full_name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.12 – Запрос 1 «Сотрудники провайдера» (api.q_provider_staff)

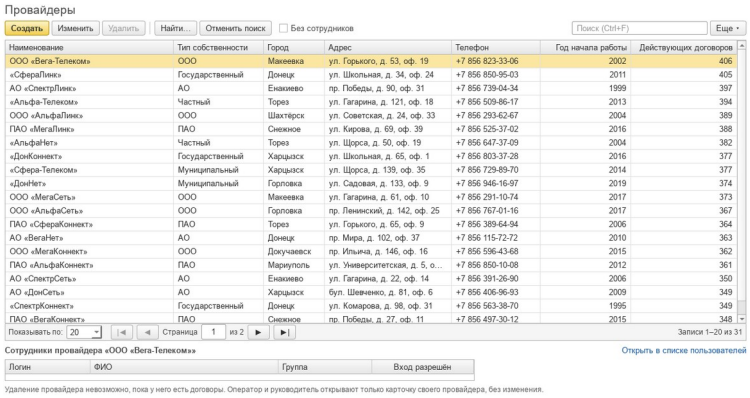


Рисунок 4.13 – Результат запроса 1 на экране «Провайдеры» (нижняя таблица)

Запрос 2 выбирает сеансы одного договора за период с тарифом и итогами по всему отбору (рисунки 4.14 и 4.15); используется на экране «Абоненты» и на вкладке «Сеансы» карточки договора.

```
SELECT s.id, s.started_at, t.code::text, t.name::text,
       s.volume_mb, s.discount_pct, s.cost,
       count(*) OVER (), sum(s.volume_mb) OVER (), sum(s.cost) OVER ()
FROM core.session s
JOIN core.contract c ON c.id = s.contract_id
JOIN ref.tariff t ON t.id = s.tariff_id
WHERE s.contract_id = p_contract_id
  AND c.provider_id = ANY (v_scope)
  AND (p_from IS NULL OR s.started_at >= p_from)
  AND (p_to IS NULL OR s.started_at < p_to + 1)
ORDER BY s.started_at DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.14 – Запрос 2 «Сеансы договора за период» (api.q_contract_sessions)

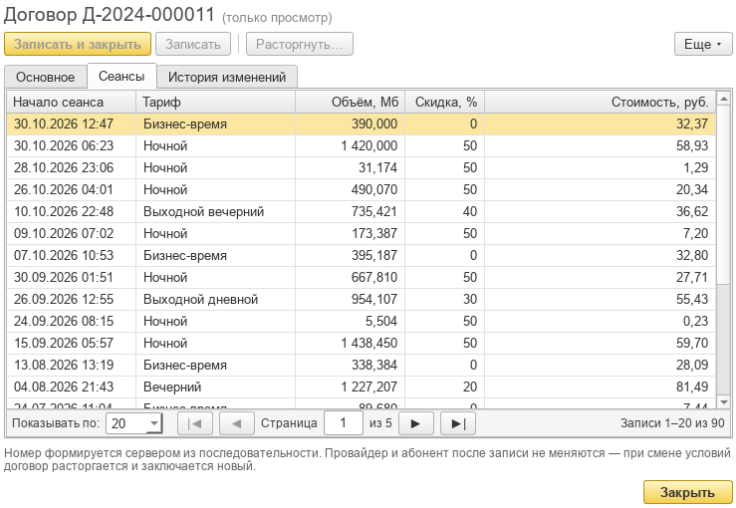


Рисунок 4.15 – Результат запроса 2: вкладка «Сеансы» карточки договора

Запрос 3 — журнал сеансов за период (условие по дате начала сеанса) с необязательными отборами по договору, IP-адресу или подсети и тарифу (рисунки 4.16 и 4.17); экран «Сеансы связи».

```
SELECT s.id, s.started_at, c.id, c.contract_no::text,
       sb.id, sb.name::text, host(c.ip_address),
       t.code::text, t.name::text, s.volume_mb, s.discount_pct,
       c.price_per_mb, s.cost,
       count(*) OVER (), sum(s.volume_mb) OVER (), sum(s.cost) OVER ()
FROM core.session s
JOIN core.contract c ON c.id = s.contract_id
JOIN core.subscriber sb ON sb.id = c.subscriber_id
JOIN ref.tariff t ON t.id = s.tariff_id
WHERE s.started_at >= p_from AND s.started_at < p_to + 1
  AND c.provider_id = ANY (v_scope)
  AND (p_contract_id IS NULL OR s.contract_id = p_contract_id)
  AND (v_ip IS NULL OR c.ip_address <<= v_ip)
  AND (p_tariff_id IS NULL OR s.tariff_id = p_tariff_id)
ORDER BY s.started_at DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.16 – Запрос 3 «Журнал сеансов за период» (api.q_sessions)

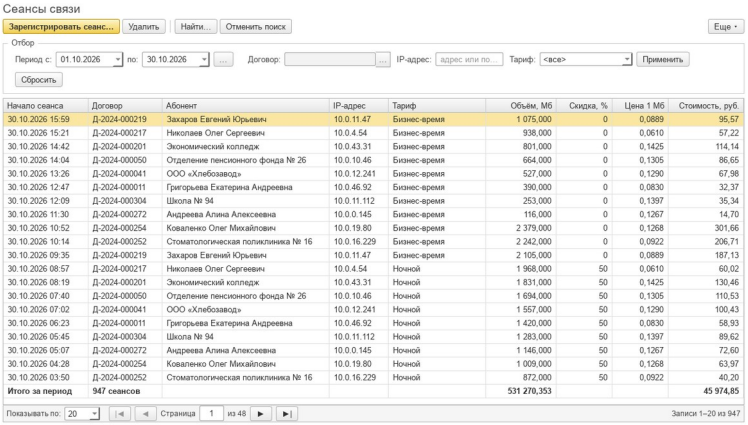


Рисунок 4.17 – Результат запроса 3 на экране «Сеансы связи»

Запрос 4 выбирает договоры, заключённые в периоде (условие по дате подключения), с абонентом, провайдером и состоянием; отборы — по абоненту, только действующие, поиск по началу номера, наименования или IP-адреса (рисунки 4.18 и 4.19); экраны «Договоры» и «Абоненты».

```
SELECT c.id, c.contract_no::text,
       CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
            ELSE '«' || p.name || '»' END,
       s.id, s.name::text, st.name::text,
       c.connected_on, c.connection_fee, c.price_per_mb,
       host(c.ip_address), c.closed_on,
       CASE WHEN c.closed_on IS NULL THEN 'Действует'
            ELSE 'Расторгнут ' || to_char(c.closed_on, 'DD.MM.YYYY') END,
       count(*) OVER ()
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN core.provider p ON p.id = c.provider_id
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
WHERE c.provider_id = ANY (v_scope)
  AND (p_from IS NULL OR c.connected_on >= p_from)
  AND (p_to IS NULL OR c.connected_on <= p_to)
  AND (p_subscriber_id IS NULL OR c.subscriber_id = p_subscriber_id)
  AND (NOT p_active_only OR c.closed_on IS NULL)
  AND (v_like IS NULL OR c.contract_no LIKE v_no_like OR lower(s.name) LIKE v_like
       OR host(c.ip_address) LIKE v_ip_like)
ORDER BY c.connected_on DESC, c.contract_no DESC
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.18 – Запрос 4 «Договоры, заключённые в периоде» (api.q_contracts)

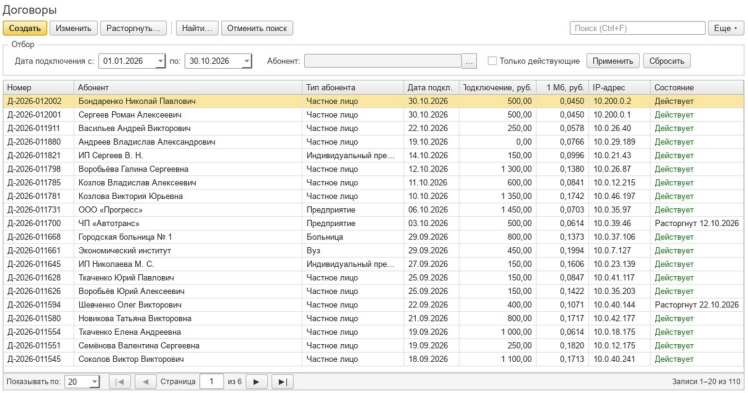


Рисунок 4.19 – Результат запроса 4 на экране «Договоры»

4.7.2 Симметричное внутреннее соединение без условия

Запросы 5–7 соединяют основную таблицу со справочниками без условий отбора и оформлены представлениями. Запрос 5 — провайдеры с типом собственности и городом (рисунки 4.20 и 4.21); экран «Мой провайдер», формы выбора провайдера.

```
CREATE OR REPLACE VIEW api.v_providers WITH (security_barrier = true) AS
SELECT p.id,
    p.name::text AS name,
    CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
        ELSE '«' || p.name || '»' END AS display_name,
    p.ownership_type_id,
    ot.name::text AS ownership_type,
    p.city_id,
    c.name::text AS city,
    p.address::text AS address,
    p.phone::text AS phone,
    p.founded_year::integer AS founded_year
FROM core.provider p
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
JOIN ref.city c ON c.id = p.city_id
WHERE p.id = ANY ((SELECT sec.scope_providers())::integer[]);
```

Рисунок 4.20 – Запрос 5 «Провайдеры» (api.v_providers)

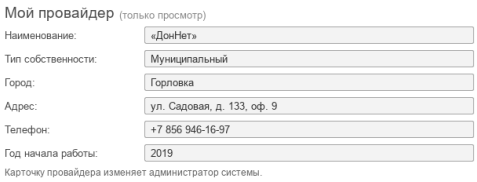


Рисунок 4.21 – Результат запроса 5 на экране «Мой провайдер»

Запрос 6 — абоненты с типом абонента и городом; оператору видны абоненты, у которых есть договор с его провайдером (рисунки 4.22 и 4.24); карточка абонента, форма выбора абонента.

```
CREATE OR REPLACE VIEW api.v_subscribers WITH (security_barrier = true) AS
SELECT s.id,
    s.name::text AS name,
    s.subscriber_type_id,
    st.name::text AS subscriber_type,
    st.is_legal,
    s.city_id,
    c.name::text AS city,
    s.address::text AS address,
    s.phone::text AS phone,
    s.row_version,
    (SELECT sec.current_group()) IS NOT DISTINCT FROM 'grp_admin'
      OR (SELECT sec.current_group()) IS NOT DISTINCT FROM 'grp_operator'
        AND NOT EXISTS (SELECT 1 FROM core.contract x
                        WHERE x.subscriber_id = s.id
                          AND x.provider_id IS DISTINCT FROM (SELECT sec.current_provider_id())) AS can_edit
FROM (SELECT s0.* FROM core.subscriber s0
    WHERE (SELECT sec.current_group()) IN ('grp_chief', 'grp_admin')
    UNION ALL
    SELECT s0.* FROM core.subscriber s0
    WHERE (SELECT sec.current_group()) NOT IN ('grp_chief', 'grp_admin')
      AND s0.id IN (SELECT v.subscriber_id FROM core.contract v
                    WHERE v.provider_id = ANY ((SELECT sec.scope_data())::integer[]))) s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city c ON c.id = s.city_id;
```

Рисунок 4.22 – Запрос 6 «Абоненты» (api.v_subscribers)

Запрос 7 — договоры с абонентом, типом абонента, провайдером и данными подключения (рисунки 4.23 и 4.25); карточка договора.

```
CREATE OR REPLACE VIEW api.v_contracts WITH (security_barrier = true) AS
SELECT c.id,
    c.contract_no::text AS contract_no,
    c.provider_id,
    CASE WHEN ot.name = upper(ot.name) THEN ot.name || ' «' || p.name || '»'
        ELSE '«' || p.name || '»' END AS provider_name,
    c.subscriber_id,
    s.name::text AS subscriber_name,
    st.name::text AS subscriber_type,
    s.phone::text AS subscriber_phone,
    c.connected_on,
    c.connection_fee,
    c.price_per_mb,
    host(c.ip_address) AS ip_address,
    c.closed_on,
    CASE WHEN c.closed_on IS NULL THEN 'Действует'
        ELSE 'Расторгнут ' || to_char(c.closed_on, 'DD.MM.YYYY') END AS state,
    c.note,
    c.row_version,
    c.connection_address::text AS connection_address,
    c.contact_phone::text AS contact_phone,
    c.contact_person::text AS contact_person
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN core.provider p ON p.id = c.provider_id
JOIN ref.ownership_type ot ON ot.id = p.ownership_type_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[]);
```

Рисунок 4.23 – Запрос 7 «Договоры» (api.v_contracts)

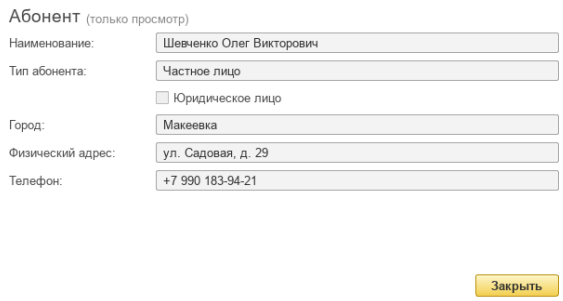


Рисунок 4.24 – Результат запроса 6: карточка абонента

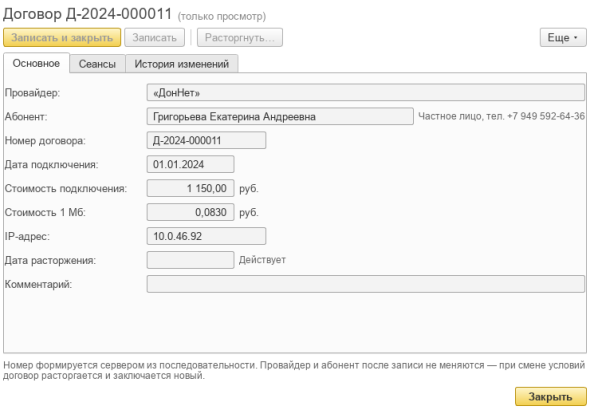


Рисунок 4.25 – Результат запроса 7: карточка договора

4.7.3 Внешние соединения и запрос на запросе

Запрос 8 присоединяет к абонентам их договоры левым внешним соединением и считает число всех и действующих договоров; абонент без договоров остаётся в результате с нулём (его видит главный аналитик). Поиск выполняется по началу наименования (LIKE с префиксом) или цифрам телефона,

видимость абонентов оператора — подзапросом IN по договорам его провайдера (рисунки 4.26 и 4.27); экран «Абоненты».

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
       count(c.id), count(c.id) FILTER (WHERE c.closed_on IS NULL),
       count(*) OVER ()
FROM (SELECT s0.* FROM core.subscriber s0 WHERE v_all
      UNION ALL
      SELECT s0.* FROM core.subscriber s0
      WHERE NOT v_all
        AND s0.id IN (SELECT v.subscriber_id FROM core.contract v WHERE v.provider_id = ANY (v_scope))) s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci ON ci.id = s.city_id
LEFT JOIN core.contract c ON c.subscriber_id = s.id
                         AND c.provider_id = ANY (v_scope)
WHERE (p_type_id IS NULL OR s.subscriber_type_id = p_type_id)
  AND (v_like IS NULL OR lower(s.name) LIKE v_like
       OR translate(s.phone, '+ ()-', '') LIKE v_phone_like)
GROUP BY s.id, s.name, s.address, s.phone, st.id, ci.id
HAVING NOT p_active_only OR count(c.id) FILTER (WHERE c.closed_on IS NULL) > 0
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.26 – Запрос 8 «Абоненты и число их договоров» (api.q_subscribers)

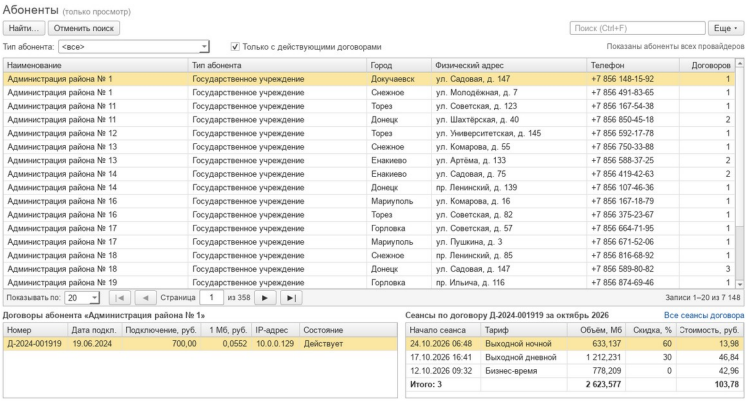


Рисунок 4.27 – Результат запроса 8 на экране «Абоненты» (главный аналитик)

Запрос 9 группирует сеансы периода по тарифу и правым внешним соединением дополняет результат всеми тарифами справочника — тариф без сеансов выводится с нулями (рисунки 4.28 и 4.29); отчёт «Трафик по тарифам».

```
SELECT t.id::integer, t.code::text, t.name::text, t.discount_pct,
       coalesce(u.sessions, 0), coalesce(u.volume_mb, 0), coalesce(u.revenue, 0),
       count(*) OVER ()
FROM (SELECT s.tariff_id, count(*) AS sessions, sum(s.volume_mb) AS volume_mb, sum(s.cost) AS revenue
      FROM core.session s
      JOIN core.contract c ON c.id = s.contract_id
      WHERE s.started_at >= p_from AND s.started_at < p_to + 1
        AND c.provider_id = ANY (v_scope)
      GROUP BY s.tariff_id) u
RIGHT JOIN ref.tariff t ON t.id = u.tariff_id
ORDER BY t.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.28 – Запрос 9 «Использование тарифов за период» (api.q_tariff_usage)

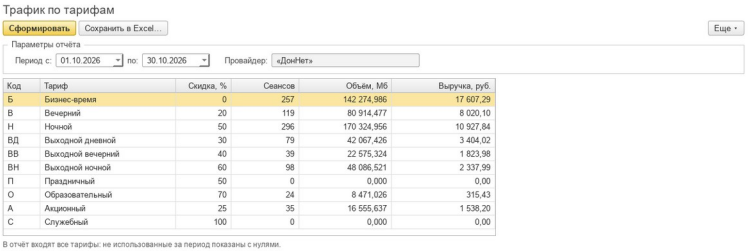


Рисунок 4.29 – Результат запроса 9 в отчёте «Трафик по тарифам»

Запрос 10 построен на запросе 5: представление провайдеров левым соединением дополняется подзапросом, который считает действующие договоры и договоры периода; провайдер без договоров остаётся в списке (рисунки 4.30 и 4.31); экран «Провайдеры».

```
SELECT v.id, v.display_name, v.ownership_type, v.city, v.address, v.phone, v.founded_year,
       coalesce(k.active, 0), coalesce(k.new_in_period, 0),
       count(*) OVER ()
FROM api.v_providers v
LEFT JOIN (
    SELECT c.provider_id,
           count(*) FILTER (WHERE c.closed_on IS NULL) AS active,
           count(*) FILTER (WHERE (p_from IS NULL OR c.connected_on >= p_from)
                            AND (p_to IS NULL OR c.connected_on <= p_to)) AS new_in_period
    FROM core.contract c
    WHERE c.provider_id = ANY (v_scope)
    GROUP BY c.provider_id) k ON k.provider_id = v.id
WHERE (v_like IS NULL OR lower(v.name) LIKE v_like)
  AND (NOT p_without_new OR k.provider_id IS NULL OR k.new_in_period = 0)
  AND (NOT p_without_staff
       OR v.id NOT IN (SELECT u.provider_id FROM sec.app_user u WHERE u.provider_id IS NOT NULL))
ORDER BY k.active DESC NULLS LAST, v.name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.30 – Запрос 10 «Провайдеры с числом договоров» (api.q_providers)

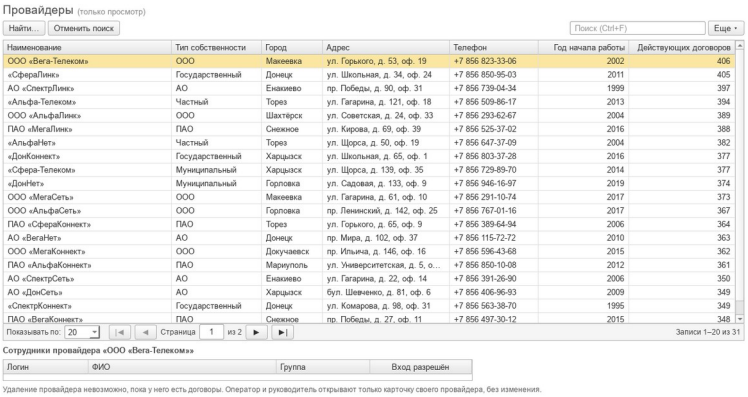


Рисунок 4.31 – Результат запроса 10 на экране «Провайдеры» (главный аналитик)

4.7.4 Итоговые запросы, запрос на запросе и запрос с подзапросом

Итоговые запросы 11–14, запрос на запросе 15 и запрос с подзапросом 16 берут данные из нескольких таблиц, условия отбора передаются параметрами функций.

Запрос 11 — итоговый без условия: считает пользователей каждой группы и всего; строка «Все пользователи» получена группировкой ROLLUP, а не отдельным запросом (рисунки 4.32 и 4.33); панель групп экрана «Пользователи».

```
CREATE OR REPLACE VIEW api.v_user_groups WITH (security_barrier = true) AS
SELECT CASE WHEN grouping(g.rolname) = 1 THEN 'Все пользователи'
            ELSE sec.group_title(g.rolname::text) END AS title,
       g.rolname::text AS group_name,
       count(*) AS users,
       CASE WHEN grouping(g.rolname) = 1 THEN 0
            ELSE array_position(ARRAY['grp_dba', 'grp_admin', 'grp_chief', 'grp_manager', 'grp_operator']::name[],
                                g.rolname) END AS ord
FROM sec.app_user u
JOIN pg_catalog.pg_roles r        ON r.rolname = u.login
JOIN pg_catalog.pg_auth_members m ON m.member = r.oid AND m.inherit_option
JOIN pg_catalog.pg_roles g        ON g.oid = m.roleid
WHERE g.rolname = ANY ((SELECT sec.visible_groups(sec.current_group()))::name[])
  AND (SELECT sec.assert_password_changed())
GROUP BY ROLLUP (g.rolname)
ORDER BY ord;
```

Рисунок 4.32 – Запрос 11 «Пользователи по группам» (api.v_user_groups)

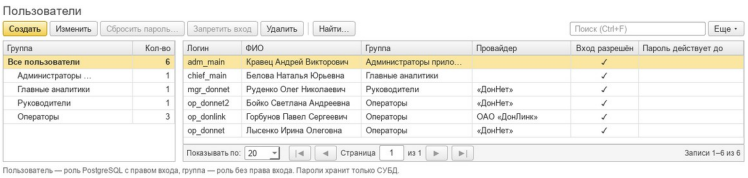


Рисунок 4.33 – Результат запроса 11: панель групп экрана «Пользователи»

Запрос 12 — итоговый с условием на данные: считает новые договоры по периодам (месяц, квартал, год) и типам абонентов; условие WHERE отбирает договоры видимых провайдеров, заключённые в периоде (рисунки 4.34 и 4.35); отчёт «Новые договоры по типам абонентов».

```
SELECT k.period, sec.period_title(p_grain, k.period), st.id::integer, st.name::text, k.contracts
FROM (SELECT date_trunc(p_grain, c.connected_on::timestamp)::date AS period, s.subscriber_type_id,
             count(*) AS contracts
      FROM core.contract c
      JOIN core.subscriber s ON s.id = c.subscriber_id
      WHERE c.provider_id = ANY (v_scope) AND c.connected_on BETWEEN p_from AND p_to
      GROUP BY 1, s.subscriber_type_id) k
JOIN ref.subscriber_type st ON st.id = k.subscriber_type_id
ORDER BY k.period, st.id;
```

Рисунок 4.34 – Запрос 12 «Новые договоры по периодам и типам абонентов» (api.report_new_contracts)

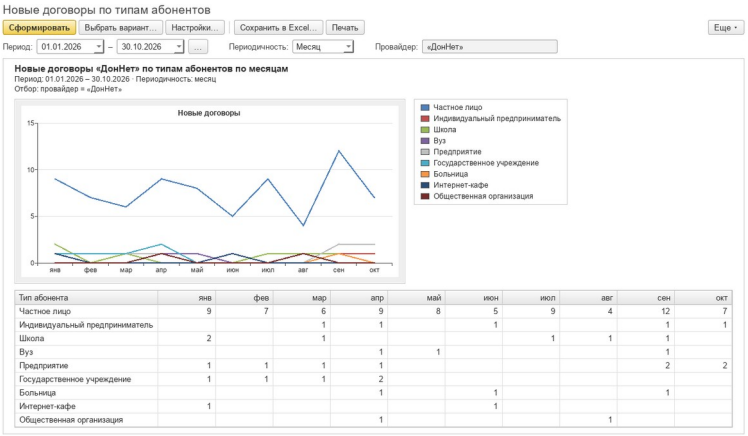


Рисунок 4.35 – Результат запроса 12 в отчёте «Новые договоры по типам абонентов»

Запрос 13 — итоговый с условием на группы: выбирает общих абонентов — тех, у кого договоры не меньше чем с заданным числом провайдеров; условие записано в HAVING (рисунки 4.36 и 4.37); экран «Общие абоненты» администратора приложения.

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, s.address::text, s.phone::text,
       count(DISTINCT c.provider_id), count(*) OVER ()
FROM core.subscriber s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci            ON ci.id = s.city_id
JOIN core.contract c        ON c.subscriber_id = s.id
GROUP BY s.id, st.id, ci.id
HAVING count(DISTINCT c.provider_id) >= p_min_providers
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.36 – Запрос 13 «Общие абоненты» (api.q_shared_subscribers)

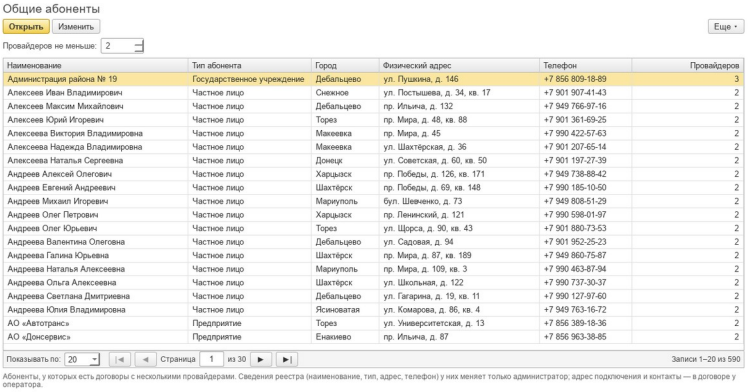


Рисунок 4.37 – Результат запроса 13 на экране «Общие абоненты»

Запрос 14 — итоговый с условием на данные и на группы: условие WHERE отбирает действующие договоры провайдера, условие HAVING — договоры, у

которых последний сеанс раньше заданного числа дней (рисунки 4.38 и 4.39); счётчик начальной страницы и список «Договоры без сеансов».

```
SELECT c.id, c.contract_no::text, s.id, s.name::text, coalesce(c.contact_phone, s.phone)::text,
       c.connected_on, max(se.started_at),
       current_date - greatest(max(se.started_at)::date, c.connected_on),
       count(*) OVER ()
FROM core.contract c
JOIN core.subscriber s    ON s.id = c.subscriber_id
LEFT JOIN core.session se ON se.contract_id = c.id
WHERE c.provider_id = ANY (v_scope) AND c.closed_on IS NULL
  AND c.connected_on < v_border
GROUP BY c.id, c.provider_id, s.id
HAVING max(se.started_at) IS NULL
    OR max(se.started_at) < v_border::timestamptz
ORDER BY max(se.started_at) NULLS FIRST, c.connected_on, c.contract_no
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.38 – Запрос 14 «Договоры без сеансов» (api.q_idle_contracts)

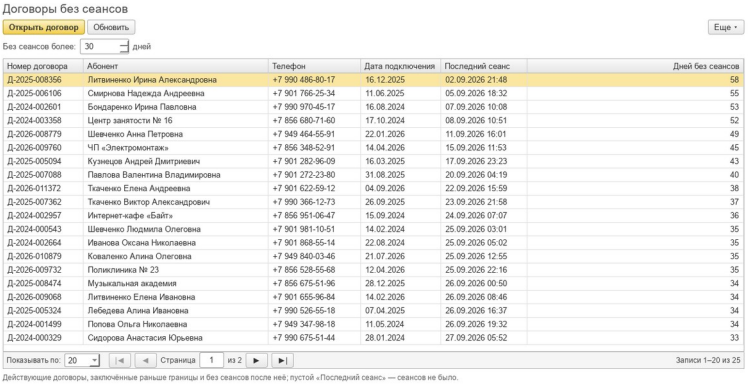


Рисунок 4.39 – Результат запроса 14 на экране «Договоры без сеансов»

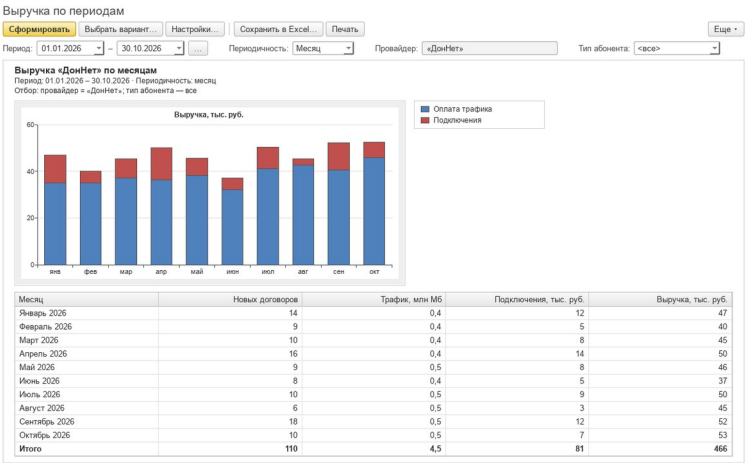
Запрос 15 — запрос на запросе по принципу итогового запроса. Внутренний запрос — представление доходных операций (запрос 20): плата за подключение и оплата трафика (рисунок 4.40); внешний группирует операции по периодам с ROLLUP и добавляет строку «Итого» (рисунки 4.41 и 4.42); отчёт «Выручка по периодам» и счётчик «Выручка за месяц» начальной страницы.

```
CREATE OR REPLACE VIEW api.v_revenue_items WITH (security_barrier = true) AS
SELECT c.provider_id,
      c.subscriber_id,
      s.subscriber_type_id,
      c.connected_on::timestamptz AS at,
      'подключение'::text AS kind,
      c.connection_fee AS amount,
      0 AS volume_mb
FROM core.contract c
JOIN core.subscriber s ON s.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[])
UNION ALL
SELECT c.provider_id,
      c.subscriber_id,
      sb.subscriber_type_id,
      se.started_at,
      'трафик'::text,
      se.cost,
      se.volume_mb
FROM core.session se
JOIN core.contract c ON c.id = se.contract_id
JOIN core.subscriber sb ON sb.id = c.subscriber_id
WHERE c.provider_id = ANY ((SELECT sec.scope_data())::integer[]);
```

Рисунок 4.40 – Запрос 20 «Доходные операции» (api.v_revenue_items) — внутренний запрос запроса 15

```
SELECT k.period, CASE WHEN k.is_total THEN 'Итого' ELSE sec.period_title(p_grain, k.period) END,
      k.new_contracts, k.volume_mb, k.connection_fees, k.revenue - k.connection_fees, k.revenue
FROM (SELECT date_trunc(p_grain, r.at)::date AS period,
            grouping(date_trunc(p_grain, r.at)::date) = 1                  AS is_total,
            count(*) FILTER (WHERE r.kind = 'подключение')                 AS new_contracts,
            coalesce(sum(r.volume_mb), 0)                                  AS volume_mb,
            coalesce(sum(r.amount) FILTER (WHERE r.kind = 'подключение'), 0)  AS connection_fees,
            coalesce(sum(r.amount), 0)                                     AS revenue
      FROM api.v_revenue_items r
      WHERE r.at >= p_from AND r.at < p_to + 1
        AND (p_provider_id IS NULL OR r.provider_id = p_provider_id)
        AND (p_subscriber_type_id IS NULL OR r.subscriber_type_id = p_subscriber_type_id)
      GROUP BY ROLLUP (date_trunc(p_grain, r.at)::date)) k
ORDER BY k.is_total, k.period;
```

Рисунок 4.41 – Запрос 15 «Выручка по периодам» (api.report_revenue)



Месяц	Новых договоров	Трафик, млн Мб	Подключения, тыс. руб.	Выручка, тыс. руб.
Январь 2026	14	0,4	12	47
Февраль 2026	9	0,4	5	40
Март 2026	10	0,4	8	45
Апрель 2026	16	0,4	14	50
Май 2026	9	0,5	8	46
Июнь 2026	8	0,4	5	37
Июль 2026	10	0,5	9	50
Август 2026	6	0,5	3	45
Сентябрь 2026	18	0,5	12	52
Октябрь 2026	10	0,5	7	53
Итого	**110**	**4,5**	**81**	**466**

Рисунок 4.42 – Результат запроса 15 в отчёте «Выручка по периодам» (руководитель)

Запрос 16 — запрос с подзапросом: отбор «Без сотрудников» в функции api.q_providers выбирает провайдеров, коды которых не входят (NOT IN) в

подзапрос по сотрудникам; значения NULL в подзапросе исключаются явно, иначе разность множеств была бы пустой (рисунки 4.43 и 4.44).

```
RETURN QUERY
SELECT v.id, v.display_name, v.ownership_type, v.city, v.address, v.phone, v.founded_year,
       coalesce(k.active, 0), coalesce(k.new_in_period, 0),
       count(*) OVER ()
FROM api.v_providers v
…
WHERE (v_like IS NULL OR lower(v.name) LIKE v_like)
  AND (NOT p_without_new OR k.provider_id IS NULL OR k.new_in_period = 0)
  AND (NOT p_without_staff
       OR v.id NOT IN (SELECT u.provider_id FROM sec.app_user u WHERE u.provider_id IS NOT NULL))
ORDER BY k.active DESC NULLS LAST, v.name
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.43 – Запрос 16 «Провайдеры без сотрудников» (api.q_providers, отбор p_without_staff)

Пельмени домашние (около 10 порций, 140–150 штук)

Тесто: мука пшеничная — 500 г; молоко — 200 мл; яйца — 2 шт. (около 120 г); растительное масло — 1 ст. л.; соль — 1 ч. л.
Фарш: свино-говяжий фарш — 500 г; репчатый лук — 1 шт.; чеснок — 2 зубчика; молоко — 2 ст. л.; зелень — половина пучка; соль и перец — по вкусу.

1. Соедините всё для теста и вымешивайте около 5 минут, пока оно не станет гладким.
2. Заверните тесто в плёнку и дайте ему полежать 40 минут.
3. Мелко нарубите лук и зелень, раздавите чеснок, смешайте с фаршем и молоком, посолите и поперчите.
4. Раскатайте тесто тонким пластом (около 1 мм), разложите фарш и слепите пельмени руками или в пельменнице.
5. Опустите пельмени в кипящую подсоленную воду и варите 6 минут после того, как вода снова закипит.

Источник: журнал «Т—Ж», t-j.ru/pelmeni

Рисунок 4.44 – Результат запроса 16: провайдеры без сотрудников (администратор приложения)

4.7.5 Итоговые запросы с итогами «всего» и «в том числе», по маске и с использованием индекса

Запрос 17 — итоговый без условия с итоговыми данными «всего» и «в том числе»: сводка по типам абонентов. Группировка ROLLUP по признаку юридического лица и типу абонента даёт строку «Всего», строки «в том числе юридические лица» и «в том числе физические лица» и типы под ними; в колонках — число абонентов, договоров всего и в том числе действующих (рисунки 4.45 и 4.46). Условий от пользователя у запроса нет, поэтому он оформлен представлением; руководителю представление показывает только абонентов своего провайдера.

```
CREATE OR REPLACE VIEW api.v_subscriber_type_totals WITH (security_barrier = true) AS
SELECT CASE WHEN grouping(st.is_legal) = 1 THEN 'Всего'
            WHEN grouping(st.name) = 1 THEN CASE WHEN st.is_legal THEN 'в том числе юридические лица'
                                                 ELSE 'в том числе физические лица' END
            ELSE st.name END                          AS title,
       2 - grouping(st.is_legal) - grouping(st.name)  AS level,
       count(DISTINCT s.id)                           AS subscribers,
       count(c.id)                                    AS contracts,
       count(c.id) FILTER (WHERE c.closed_on IS NULL) AS contracts_active,
       row_number() OVER (ORDER BY grouping(st.is_legal) DESC, st.is_legal DESC,
                                   grouping(st.name) DESC, min(st.id)) AS sort_order
FROM (SELECT s0.id, s0.subscriber_type_id FROM core.subscriber s0
      WHERE (SELECT sec.current_group()) IS NOT DISTINCT FROM 'grp_chief'
      UNION ALL
      SELECT s0.id, s0.subscriber_type_id FROM core.subscriber s0
      WHERE (SELECT sec.current_group()) IS DISTINCT FROM 'grp_chief'
        AND s0.id IN (SELECT v.subscriber_id FROM core.contract v
                      WHERE v.provider_id = ANY ((SELECT sec.scope_data())::integer[]))) s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
LEFT JOIN core.contract c   ON c.subscriber_id = s.id
                           AND c.provider_id = ANY ((SELECT sec.scope_data())::integer[])
GROUP BY ROLLUP (st.is_legal, st.name)
ORDER BY sort_order;
```

Рисунок 4.45 – Запрос 17 «Абоненты и договоры по типам абонентов» (api.v_subscriber_type_totals)

	Тип абонента character varying	Абонентов bigint	Договоров bigint	В т. ч. действующих bigint
1	Всего	289	409	374
2	в том числе юридические лица	76	106	96
3	Школа	10	12	12
4	Колледж	3	3	3
5	Вуз	6	11	11
6	Предприятие	23	32	27
7	Государственное учреждение	13	20	18
8	Библиотека	1	1	1
9	Больница	5	7	6
10	Интернет-кафе	8	10	8
11	Общественная организация	7	10	10
12	в том числе физические лица	213	303	278
13	Частное лицо	193	280	256
14	Индивидуальный предприниматель	20	23	22

Рисунок 4.46 – Результат запроса 17 в pgAdmin (руководитель)

Запрос 18 — итоговый с условием на данные по маске: трафик абонентов, наименование которых подходит под маску, за период. Символ «*» в маске обозначает любые символы, «?» — один символ; функция sec.like_mask переводит маску в шаблон LIKE и экранирует служебные символы шаблона во вводе пользователя (рисунки 4.47 и 4.48, маска «*школа*»).

```
SELECT s.id, s.name::text, st.name::text,
       count(DISTINCT c.id), count(*), sum(se.volume_mb), sum(se.cost),
       count(*) OVER (), sum(count(*)) OVER (), sum(sum(se.volume_mb)) OVER (), sum(sum(se.cost)) OVER ()
FROM core.subscriber s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN core.contract c        ON c.subscriber_id = s.id
JOIN core.session se        ON se.contract_id = c.id AND se.provider_id = c.provider_id
WHERE lower(s.name) LIKE v_like
  AND c.provider_id = ANY (v_scope)
  AND se.started_at >= p_from AND se.started_at < p_to + 1
GROUP BY s.id, st.id
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.47 – Запрос 18 «Трафик абонентов по маске наименования» (api.q_subscriber_traffic)

	Абонент text	Тип абонента text	Договоров bigint	Сеансов bigint	Трафик, Мб numeric	Сумма, руб. numeric	Всего абонентов bigint	Итого Мб numeric	Итого, руб. numeric
1	Школа № 126	Школа	1	4	3679.956	486.55	10	96877.504	6081.97
2	Школа № 36	Школа	1	20	6555.294	286.94	10	96877.504	6081.97
3	Школа № 43	Школа	1	25	10602.203	475.35	10	96877.504	6081.97
4	Школа № 44	Школа	1	23	11003.514	300.73	10	96877.504	6081.97
5	Школа № 61	Школа	2	34	23399.477	1546.82	10	96877.504	6081.97
6	Школа № 62	Школа	1	22	8081.538	437.19	10	96877.504	6081.97
7	Школа № 76	Школа	2	32	9961.446	400.90	10	96877.504	6081.97
8	Школа № 85	Школа	1	23	8767.980	1035.29	10	96877.504	6081.97
9	Школа № 94	Школа	1	23	13933.066	1071.39	10	96877.504	6081.97
10	Школа № 97	Школа	1	1	893.030	40.81	10	96877.504	6081.97

Рисунок 4.48 – Результат запроса 18 в pgAdmin (руководитель, маска «*школа*»)

Запрос 19 — итоговый с условием на данные по договору и периоду: трафик договора по месяцам со строкой «Итого» (рисунки 4.49 и 4.50). Условие выполняется с использованием индекса «договор, дата сеанса»: в каждой квартальной секции периода читаются только страницы сеансов договора (рисунок 4.51). Тот же вызов при запрете индексного доступа (параметры enable_indexscan и enable_bitmapscan в транзакции) выполняется без использования индекса и читает секции периода целиком (рисунок 4.52). Результат одинаков, но без индекса прочитано 2 111 страниц вместо 94, а время выполнения выросло с 1,4 до 21,0 мс.

```
SELECT k.period, CASE WHEN k.period IS NULL THEN 'Итого' ELSE sec.period_title('month', k.period) END,
       k.sessions, k.volume_mb, round(k.full_cost, 2), round(k.full_cost, 2) - k.cost, k.cost
FROM (SELECT date_trunc('month', se.started_at)::date AS period,
             count(*) AS sessions, sum(se.volume_mb) AS volume_mb,
             sum(se.volume_mb * c.price_per_mb) AS full_cost, sum(se.cost) AS cost
      FROM core.session se
      JOIN core.contract c ON c.id = se.contract_id AND c.provider_id = se.provider_id
      WHERE se.contract_id = p_contract_id
        AND se.started_at >= p_from AND se.started_at < p_to + 1
        AND c.provider_id = ANY (v_scope)
      GROUP BY ROLLUP (date_trunc('month', se.started_at)::date)
      HAVING count(*) > 0) k
ORDER BY k.period NULLS LAST;
```

Рисунок 4.49 – Запрос 19 «Трафик договора по месяцам» (api.report_contract_months)

	Месяц text	Сеансов bigint	Трафик, Мб numeric	Без скидки, руб. numeric	Скидка, руб. numeric	К оплате, руб. numeric
1	Январь 2026	2	124.014	16.18	6.50	9.68
2	Февраль 2026	2	77.363	10.10	0.05	10.05
3	Март 2026	5	1486.071	193.93	101.39	92.54
4	Апрель 2026	4	3270.210	426.76	89.73	337.03
5	Май 2026	1	25.883	3.38	2.03	1.35
6	Июнь 2026	3	1071.544	139.84	82.70	57.14
7	Июль 2026	3	1408.966	183.87	34.13	149.74
8	Август 2026	2	1372.497	179.11	89.55	89.56
9	Сентябрь 2026	1	93.499	12.20	6.10	6.10
10	Итого	23	8930.047	1165.37	412.18	753.19

Рисунок 4.50 – Результат запроса 19 в pgAdmin (оператор)

```
->  Append  (cost=0.28..171.61 rows=36 width=35) (actual rows=23.00 loops=1)
      Buffers: shared hit=29
      Subplans Removed: 14
      ->  Bitmap Heap Scan on session_2026_q1 se_1  (cost=4.38..30.27 rows=7 width=28) (actual rows=9.00 loops=1)
            Recheck Cond: ((contract_id = 50) AND (started_at >= '2026-01-01'::date) AND (started_at < '2026-10-01'::date))
            Heap Blocks: exact=9
            Buffers: shared hit=11
            ->  Bitmap Index Scan on session_2026_q1_contract_id_started_at_idx  (cost=0.00..4.38 rows=7 width=0) (actual rows=9.00 loops=1)
                  Index Cond: ((contract_id = 50) AND (started_at >= '2026-01-01'::date) AND (started_at < '2026-10-01'::date))
                  Index Searches: 1
                  Buffers: shared hit=2
```

Рисунок 4.51 – План запроса 19 с использованием индекса (фрагмент: первая секция периода)

```
->  Append  (cost=0.00..13430.56 rows=36 width=35) (actual rows=23.00 loops=1)
      Buffers: shared hit=2046
      Subplans Removed: 14
      ->  Seq Scan on session_2026_q1 se_1  (cost=0.00..1768.27 rows=7 width=28) (actual rows=9.00 loops=1)
            Filter: ((started_at >= '2026-01-01'::date) AND (started_at < '2026-10-01'::date) AND (contract_id = 50))
            Rows Removed by Filter: 65835
            Buffers: shared hit=616
```

Рисунок 4.52 – План запроса 19 без использования индекса (фрагмент: первая секция периода)

4.7.6 Объединение и запросы с подзапросами

Запрос 20 — запрос с объединением: представление доходных операций (рисунок 4.40) объединяет оператором UNION ALL плату за подключение (дата заключения договора) и оплату трафика (начало сеанса); совпадающие суммы являются разными операциями и не удаляются (рисунок 4.53). Представление служит основой отчётов о выручке — запросов 15 и 22.

	Дата и время text	Операция text	Сумма, руб. numeric (12,2)	Трафик, Мб numeric
1	30.10.2026 00:00	подключение	500.00	0
2	30.10.2026 00:00	подключение	500.00	0
3	22.10.2026 00:00	подключение	250.00	0
4	19.10.2026 00:00	подключение	0.00	0
5	14.10.2026 00:00	подключение	150.00	0
6	12.10.2026 00:00	подключение	1300.00	0
7	11.10.2026 00:00	подключение	600.00	0
8	10.10.2026 00:00	подключение	1350.00	0
9	06.10.2026 00:00	подключение	1450.00	0
10	03.10.2026 00:00	подключение	500.00	0
11	30.10.2026 15:59	трафик	95.57	1075.000
12	30.10.2026 15:21	трафик	57.22	938.000
13	30.10.2026 14:42	трафик	114.14	801.000
14	30.10.2026 14:04	трафик	86.65	664.000
15	30.10.2026 13:26	трафик	67.98	527.000

Рисунок 4.53 – Результат запроса 20 в pgAdmin (руководитель)

Запрос 21 — запрос с подзапросом IN: абоненты, у которых за период были сеансы по выбранному тарифу. Подзапрос выбирает абонентов договоров с такими сеансами; условие IN проверяет только факт, поэтому абонент не повторяется по числу сеансов, как при соединении (рисунки 4.54 и 4.55). Для льготного тарифа «Образовательный» результат содержит только школы, колледжи и вузы.

```
SELECT s.id, s.name::text, st.name::text, ci.name::text, count(*), count(*) OVER ()
FROM core.subscriber s
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
JOIN ref.city ci            ON ci.id = s.city_id
JOIN core.contract c        ON c.subscriber_id = s.id AND c.provider_id = ANY (v_scope)
WHERE s.id IN (SELECT x.subscriber_id
               FROM core.session se
               JOIN core.contract x ON x.id = se.contract_id AND x.provider_id = se.provider_id
               WHERE se.tariff_id = p_tariff_id
                 AND se.started_at >= p_from AND se.started_at < p_to + 1
                 AND se.provider_id = ANY (v_scope))
GROUP BY s.id, st.id, ci.id
ORDER BY s.name, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.54 – Запрос 21 «Абоненты, пользовавшиеся тарифом» (api.q_tariff_subscribers)

	Абонент text	Тип абонента text	Город text	Договоров bigint	Всего абонентов bigint
1	Горный институт	Вуз	Дебальцево	3	16
2	Горный колледж	Колледж	Ясиноватая	1	16
3	Музыкальная академия	Вуз	Донецк	3	16
4	Педагогический университет	Вуз	Снежное	1	16
5	Строительная академия	Вуз	Дебальцево	2	16
6	Школа № 36	Школа	Снежное	1	16
7	Школа № 43	Школа	Горловка	1	16
8	Школа № 44	Школа	Донецк	1	16
9	Школа № 61	Школа	Шахтёрск	2	16
10	Школа № 62	Школа	Шахтёрск	1	16
11	Школа № 76	Школа	Ясиноватая	2	16
12	Школа № 85	Школа	Горловка	1	16
13	Школа № 94	Школа	Донецк	1	16
14	Экономический институт	Вуз	Торез	1	16
15	Экономический колледж	Колледж	Докучаевск	1	16
16	Экономический колледж	Колледж	Донецк	1	16

Рисунок 4.55 – Результат запроса 21 в pgAdmin (руководитель, тариф «Образовательный»)

Запрос 22 — запрос с подзапросом CASE: доли провайдеров в выручке. Внутренний запрос считает выручку провайдера, его место и общий итог, внешний относит провайдера к группе крупнейших или к «Прочим» выражением CASE и вычисляет долю группы (рисунки 4.56 и 4.57); отчёт главного аналитика.

```
WITH x AS (
    SELECT p.id, p.display_name, k.contracts, r.volume_mb, r.revenue,
           least(row_number() OVER (ORDER BY r.revenue DESC NULLS LAST, p.id), v_top + 1)::integer AS grp,
           sum(r.revenue) OVER ()                                          AS total
    FROM api.v_providers p
    LEFT JOIN (SELECT i.provider_id, sum(i.volume_mb) AS volume_mb, sum(i.amount) AS revenue
               FROM api.v_revenue_items i
               WHERE i.at >= p_from AND i.at < p_to + 1
                 AND (p_subscriber_type_id IS NULL OR i.subscriber_type_id = p_subscriber_type_id)
               GROUP BY i.provider_id) r ON r.provider_id = p.id
    LEFT JOIN (SELECT c.provider_id, count(*) AS contracts
               FROM core.contract c
               WHERE c.connected_on <= p_to AND (c.closed_on IS NULL OR c.closed_on > p_to)
                 AND (p_subscriber_type_id IS NULL
                      OR c.subscriber_id IN (SELECT s.id FROM core.subscriber s
                                             WHERE s.subscriber_type_id = p_subscriber_type_id))
               GROUP BY c.provider_id) k ON k.provider_id = p.id),
t AS (
    SELECT CASE WHEN grouping(x.grp) = 1 THEN v_top + 2 ELSE x.grp END AS ord,
           CASE WHEN grouping(x.grp) = 1 THEN 'Итого'
                WHEN x.grp <= v_top THEN min(x.display_name)
                ELSE 'Прочие (' || sec.count_title(count(*), 'провайдер', 'провайдера', 'провайдеров') || ')'
           END AS provider_group,
           count(*) AS providers, sum(x.contracts)::bigint AS contracts,
           sum(x.volume_mb) AS volume_mb, sum(x.revenue) AS revenue,
           round(100 * sum(x.revenue) / nullif(max(x.total), 0), 1) AS share_pct,
           grouping(x.grp) = 1 AS is_total
    FROM x
    GROUP BY ROLLUP (x.grp))
SELECT t.ord, t.provider_group, t.providers, NULL, NULL,
       t.contracts, t.volume_mb, t.revenue, t.share_pct, t.is_total
FROM t
```

Рисунок 4.56 – Запрос 22 «Доли провайдеров в выручке» (api.report_provider_shares, без рядов диаграммы)

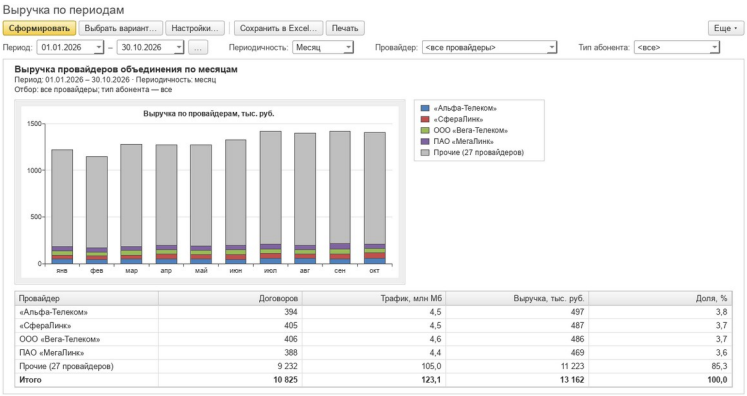


Рисунок 4.57 – Результат запроса 22 в отчёте «Выручка по периодам» (главный аналитик)

Запрос 23 — запрос с подзапросом и операциями над итоговыми данными: абоненты, трафик которых за период выше среднего трафика абонента провайдера. Внутренний запрос считает трафик каждого абонента, скалярный подзапрос — среднее этих итогов, внешний отбирает абонентов выше среднего и вычисляет превышение в процентах (рисунки 4.58 и 4.59).

```
WITH t AS (
    SELECT c.subscriber_id, sum(se.volume_mb) AS volume_mb
    FROM core.session se
    JOIN core.contract c ON c.id = se.contract_id AND c.provider_id = se.provider_id
    WHERE se.started_at >= p_from AND se.started_at < p_to + 1
      AND c.provider_id = ANY (v_scope)
    GROUP BY c.subscriber_id)
SELECT s.id, s.name::text, st.name::text, t.volume_mb,
       round((SELECT avg(t0.volume_mb) FROM t t0), 3),
       round((t.volume_mb / (SELECT avg(t0.volume_mb) FROM t t0) - 1) * 100, 1),
       count(*) OVER ()
FROM t
JOIN core.subscriber s      ON s.id = t.subscriber_id
JOIN ref.subscriber_type st ON st.id = s.subscriber_type_id
WHERE t.volume_mb > (SELECT avg(t0.volume_mb) FROM t t0)
ORDER BY t.volume_mb DESC, s.id
LIMIT least(greatest(p_limit, 1), 200) OFFSET greatest(p_offset, 0);
```

Рисунок 4.58 – Запрос 23 «Абоненты с трафиком выше среднего» (api.q_subscribers_above_avg)

	Абонент text	Тип абонента text	Трафик, Мб numeric	Среднее, Мб numeric	Выше среднего, % numeric	Всего абонентов bigint
1	Отделение пенсионного фонда № 2	Государственное ...	19587.693	5255.709	272.7	108
2	Кузнецов Андрей Дмитриевич	Частное лицо	18675.749	5255.709	255.3	108
3	Смирнов Юрий Олегович	Частное лицо	16556.587	5255.709	215.0	108
4	Горный институт	Вуз	16096.323	5255.709	206.3	108
5	Козлова Марина Михайловна	Частное лицо	15372.895	5255.709	192.5	108
6	Алексеева Виктория Владимировна	Частное лицо	15194.847	5255.709	189.1	108
7	Коваленко Наталья Ивановна	Частное лицо	15194.192	5255.709	189.1	108
8	Кравченко Анна Владимировна	Частное лицо	15053.145	5255.709	186.4	108
9	Попова Оксана Петровна	Частное лицо	14257.180	5255.709	171.3	108
10	Борисов Владислав Сергеевич	Частное лицо	14236.051	5255.709	170.9	108
11	Центр занятости № 16	Государственное ...	14141.021	5255.709	169.1	108
12	Григорьева Екатерина Андреевна	Частное лицо	13728.025	5255.709	161.2	108
13	Музыкальная академия	Вуз	13335.992	5255.709	153.7	108
14	Бондаренко Александр Михайлович	Частное лицо	13017.553	5255.709	147.7	108
15	ПАО «Хлебозавод»	Предприятие	11875.525	5255.709	126.0	108
16	Михайлова Наталья Дмитриевна	Частное лицо	11617.813	5255.709	121.1	108
17	Степанова Марина Олеговна	Частное лицо	11610.975	5255.709	120.9	108
18	Иванова Оксана Николаевна	Частное лицо	11579.657	5255.709	120.3	108

Рисунок 4.59 – Результат запроса 23 в pgAdmin (руководитель)

Корректность запросов проверена автоматическими тестами (раздел 6): результат каждого запроса сравнивается с независимым расчётом, проверяются видимость строк для каждой роли, итоги, постраничный вывод и поиск.

4.8 Создание представлений и хранимых процедур, функций

Клиентское приложение обращается к данным только через схему api: семь представлений (запросы без параметров) и функции (запросы с параметрами, изменение данных, управление пользователями, сведения для начальной страницы и администратора БД). Представления с данными предметной области создаются с параметром security_barrier и ограничивают строки условием на массив видимых провайдеров; функции оформлены по единому шаблону (рисунок 4.60): SECURITY DEFINER, фиксированные search_path и часовой пояс, проверка группы первой операцией, проверка параметров с понятным пользователю сообщением, выборка с ограничением страницы. Перечень объектов схемы api, кроме запросов подраздела 4.7, с группами, которым они доступны, приведён в таблице 4.3 (О — оператор, Р — руководитель, ГА — главный аналитик, АП — администратор приложения, АБД — администратор БД).

```
CREATE OR REPLACE FUNCTION api.q_shared_subscribers(
    p_min_providers integer DEFAULT 2,
    p_limit integer DEFAULT 20,
    p_offset integer DEFAULT 0)
RETURNS TABLE (
    id integer, name text, subscriber_type text, city text, address text, phone text,
    providers bigint, total_count bigint)
LANGUAGE plpgsql STABLE SECURITY DEFINER
SET search_path = pg_catalog, pg_temp
SET TimeZone = 'Europe/Moscow'
AS $$
BEGIN
    PERFORM sec.require_group(ARRAY['grp_admin']);
    IF p_min_providers IS NULL OR p_min_providers < 2 THEN
        RAISE EXCEPTION 'Общий абонент — абонент не меньше чем двух провайдеров' USING ERRCODE = 'P0001';
    END IF;
    …
END $$;
```

Рисунок 4.60 – Функция-список по единому шаблону (api.q_shared_subscribers)

Таблица 4.3 – Представления и функции схемы api, кроме запросов

Объект	Вид	Назначение	Группы
api.v_manageable_groups	представление	Группы, учётные записи которых создаёт и блокирует текущий администратор	АП, АБД
api.change_own_password	функция	Смена своего пароля	О, Р, ГА, АП
api.contract_create	функция	Новый договор своего провайдера с абонентом	О
api.contract_update	функция	Изменение договора своего провайдера	О
api.db_connections	функция	Подключения к базе	АБД
api.db_session_partitions_ensure	функция	Создаёт секции сеансов текущего и следующего года, если их нет	АБД
api.db_tables	функция	Состояние таблиц проекта	АБД
api.home_todo	функция	Начальная страница, блок «Текущие дела»	О, Р, ГА, АП, АБД
api.lookup_subscriber_types	функция	Список выбора «Тип абонента»	О, Р, ГА, АП
api.lookup_tariffs	функция	Список выбора «Тариф»	О, Р, ГА, АП
api.q_change_log	функция	Журнал изменений за период	АБД
api.q_change_log_fields	функция	Поля записи журнала изменений	АБД
api.q_users	функция	Пользователи системы видимых групп	АП, АБД
api.subscriber_delete	функция	Удаление абонента вместе с его договорами и сеансами	О
api.subscriber_find_exact	функция	Абонент реестра с точно таким наименованием	О
api.subscriber_update	функция	Изменение сведений абонента в реестре	О, АП
api.user_create	функция	Новый пользователь	АП, АБД
api.user_reset_password	функция	Сброс пароля сотрудника управляемой группы	АП, АБД
api.user_set_login	функция	Разрешить или запретить вход пользователю управляемой группы	АП, АБД
api.whoami	функция	Текущий пользователь	О, Р, ГА, АП, АБД

Функции изменения данных (api.contract_create, api.contract_update, api.subscriber_update, api.subscriber_delete) проверяют права на конкретную строку, используют оптимистическую блокировку по версии строки row_version

и возвращают новую версию; при одновременном изменении второй пользователь получает ошибку 40001 и перечитывает запись.

[ЗАГЛУШКА: модифицируемое представление «абонент + договор» будет добавлено после реализации]

5 РАЗРАБОТКА КЛИЕНТСКОГО ПРИЛОЖЕНИЯ

Одна программа обслуживает все роли. После входа сервер сообщает группу пользователя функцией api.whoami, и главное окно строит панель разделов по группе (таблица 5.1). Инструменты, которые роли не нужны, не показываются; руководитель и главный аналитик работают с теми же страницами, что и оператор, в режиме только просмотра — отдельных страниц одной тематики для разных ролей нет. Скрытие инструментов на клиенте — удобство; запрет выполняет сервер.

Таблица 5.1 – Разделы главного окна по группам пользователей

Группа	Разделы и формы
Оператор	Главное; Абоненты и договоры (абоненты, договоры); Сеансы связи (журнал сеансов); Мой провайдер
Руководитель	Главное; Абоненты и договоры, Сеансы связи — просмотр; Статистика (выручка, трафик по тарифам, новые договоры, крупнейшие абоненты); Мой провайдер
Главный аналитик	Главное; Провайдеры; Абоненты и договоры, Сеансы связи — просмотр; Статистика с отбором по провайдеру
Администратор приложения	Главное; Администрирование (пользователи); Справочники (типы собственности, города, типы абонентов, тарифы, общие абоненты); Провайдеры
Администратор БД	Главное; Администрирование (пользователи, журнал изменений); База данных (таблицы, подключения)

Начальная страница у всех ролей содержит сведения о пользователе и подключении, описание возможностей роли и блок «Текущие дела» — счётчики, которые считает функция api.home_todo; щелчок по счётчику открывает соответствующий список или отчёт. Числа счётчиков получены теми же функциями, что и открываемые списки, поэтому счётчик и список не расходятся.

[ЗАГЛУШКА: формы создания и изменения абонентов, договоров, сеансов, провайдеров и справочников будут описаны после реализации]

5.1 Формы и компоненты для работы в роли «Оператор»

Начальная страница оператора показана на рисунке 3.4 (подраздел 3.2): счётчики «Договоры без сеансов более 30 дней», «Сеансов зарегистрировано

сегодня», «Договоров заключено на этой неделе», «Договоры с наступившей датой расторжения».

Страница «Абоненты» — составная форма «один-ко-многим» (рисунок 5.1): верхняя таблица — абоненты с поиском по началу наименования или цифрам телефона и отбором по типу абонента и наличию действующих договоров (запрос 8), средняя — договоры выбранного абонента у своего провайдера (запрос 4), нижняя — сеансы выбранного договора с итогами (запрос 2). Двойной щелчок открывает карточку абонента или договора (рисунки 4.24 и 4.25).

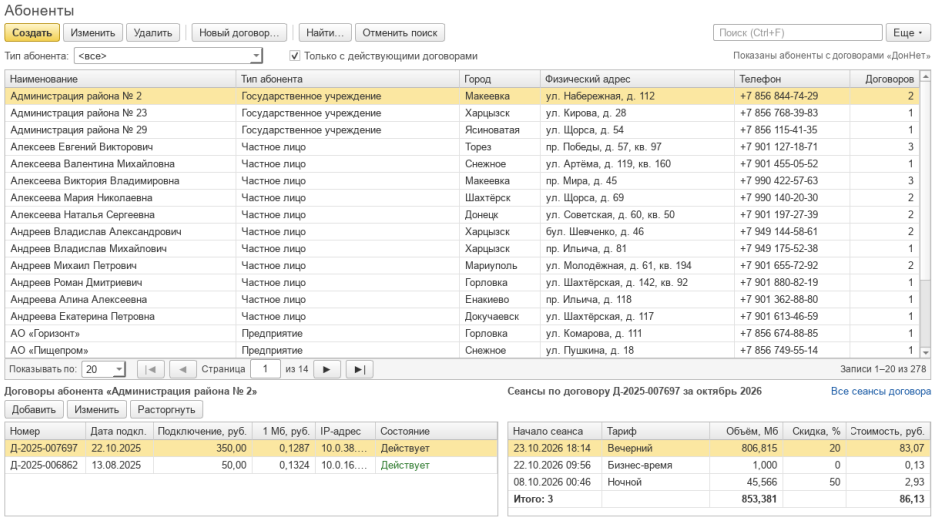


Рисунок 5.1 – Составная форма «Абоненты»: абонент → договоры → сеансы (оператор)

Страница «Договоры» выводит договоры, заключённые в периоде, с отбором по абоненту (форма выбора) и признаком «Только действующие» (рисунок 4.19); страница «Сеансы связи» — журнал сеансов за период с отбором по договору, IP-адресу или подсети и тарифу и строкой «Итого за период» (рисунок 4.17). Период задаётся полями дат или меню «…» (сегодня, эта неделя, этот месяц, прошлый месяц, этот год). Страница «Мой провайдер» — карточка своего провайдера и список его сотрудников (рисунок 4.21). Список «Договоры без сеансов» открывается счётчиком начальной страницы (рисунок 4.39).

5.2 Формы и компоненты для работы в роли «Руководитель»

Руководитель провайдера видит данные своего провайдера в режиме только просмотра: в заголовке формы выводится пометка «(только просмотр)», кнопки изменения не показываются (рисунок 5.2). Начальная страница руководителя содержит счётчики выручки и новых договоров за текущий месяц, действующих договоров и договоров без сеансов (рисунок 5.3). Раздел «Статистика» описан в подразделе 5.6.

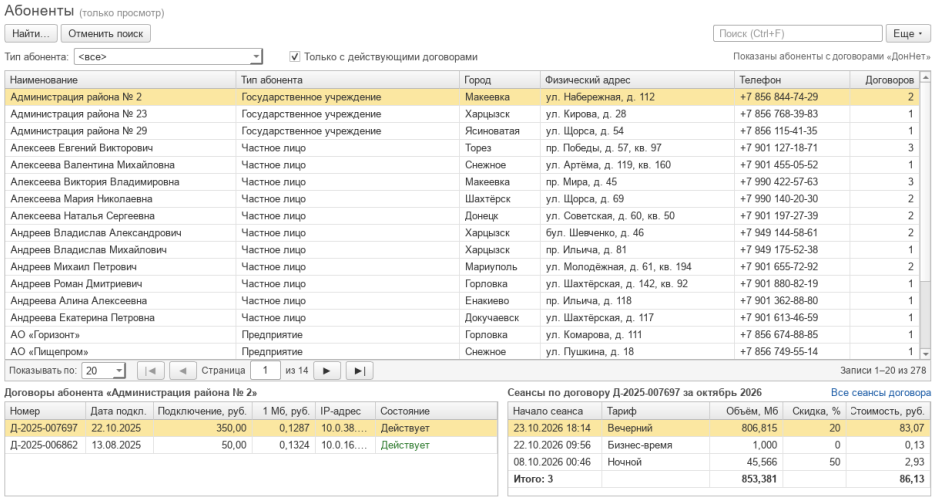


Рисунок 5.2 – Страница «Абоненты» в режиме просмотра (руководитель)

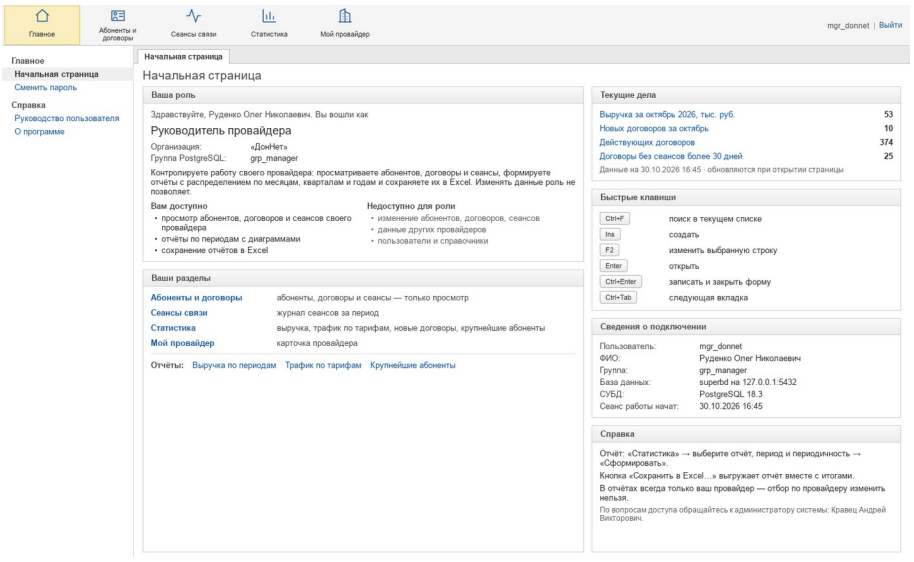


Рисунок 5.3 – Начальная страница руководителя провайдера

5.3 Формы и компоненты для работы в роли «Главный аналитик»

Главный аналитик объединения видит данные всех провайдеров: раздел «Провайдеры» со списком провайдеров и числом действующих и новых договоров (рисунок 4.31), абонентов всего реестра, включая абонентов без договоров (рисунок 4.27), журнал сеансов по всем провайдерам с отбором по провайдеру. В отчётах «Статистики» доступен отбор по провайдеру; без отбора отчёт о выручке показывает доли провайдеров (рисунок 4.57). Начальная страница главного аналитика показана на рисунке 5.4.

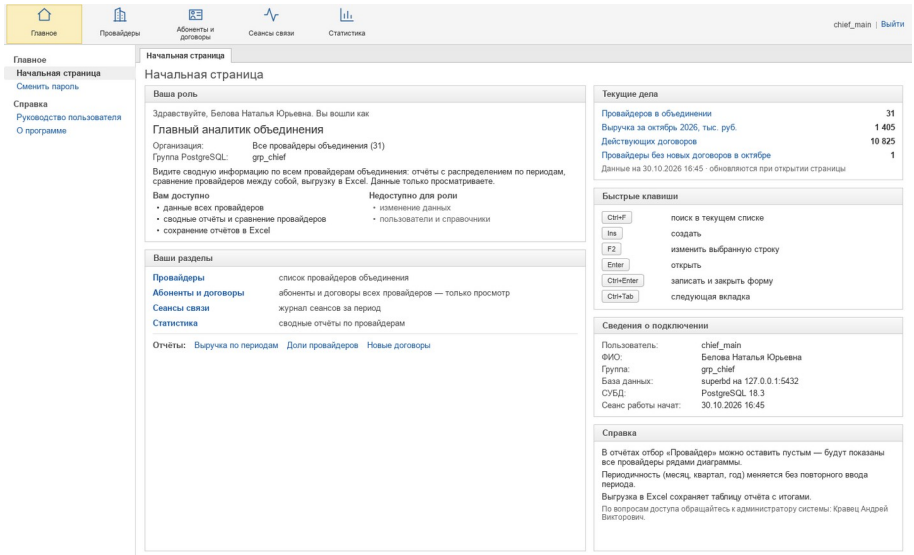


Рисунок 5.4 – Начальная страница главного аналитика

5.4 Формы и компоненты для работы в роли «Администратор приложения»

Администратор приложения ведёт учётные записи операторов, руководителей и главных аналитиков на странице «Пользователи» (рисунок 4.33): слева — группы с числом пользователей (запрос 11), справа — пользователи выбранной группы с логином, ФИО, провайдером, признаком «Вход разрешён» и сроком действия пароля. Команды страницы:

– «Создать» — форма «Пользователь (создание)»: логин, ФИО, группа, провайдер (обязателен для оператора и руководителя), признак «Вход

разрешён»; после записи сервер возвращает временный пароль, который показывается один раз;

– «Запретить вход» / «Разрешить вход» — функция api.user_set_login;

– «Сбросить пароль…» — подтверждение (рисунок 5.5) и окно с временным паролем на сутки; до смены временного пароля пользователю доступна только форма «Смена пароля» (рисунок 5.6).

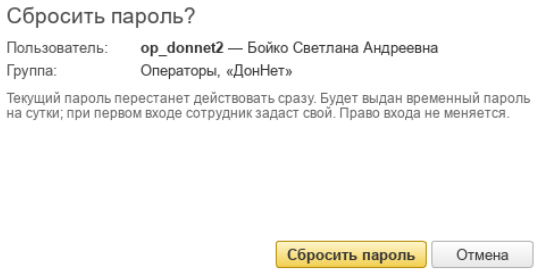


Рисунок 5.5 – Подтверждение сброса пароля

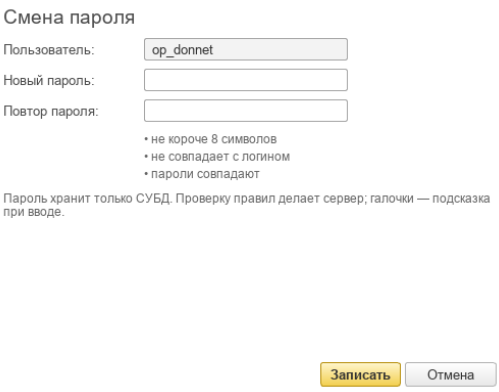


Рисунок 5.6 – Форма «Смена пароля»

В разделе «Провайдеры» администратор видит всех провайдеров и сотрудников выбранного провайдера (рисунок 4.13); отбор «Без сотрудников» показывает провайдеров, которым ещё не назначены сотрудники (рисунок 4.44). Список «Общие абоненты» раздела «Справочники» выводит абонентов, обслуживаемых несколькими провайдерами, сведения которых меняет только администратор (рисунок 4.37). Начальная страница администратора приложения показана на рисунке 5.7.

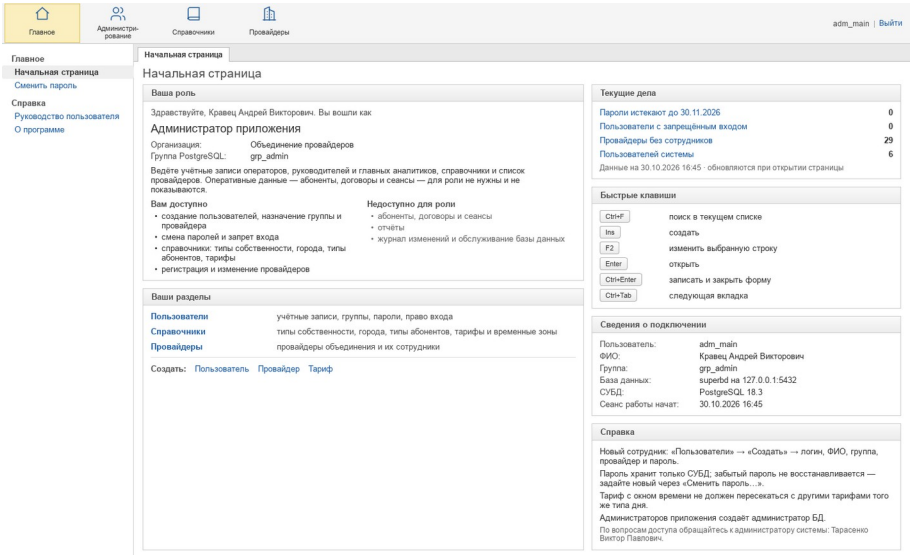


Рисунок 5.7 – Начальная страница администратора приложения

5.5 Формы и компоненты для работы в роли «Администратор БД»

Администратор базы данных создаёт и блокирует учётные записи администраторов приложения на той же странице «Пользователи» и обслуживает базу данных. Начальная страница показывает размер данных, число подключений, изменений в журнале за сегодня и таблиц без свежей статистики (рисунок 5.8).

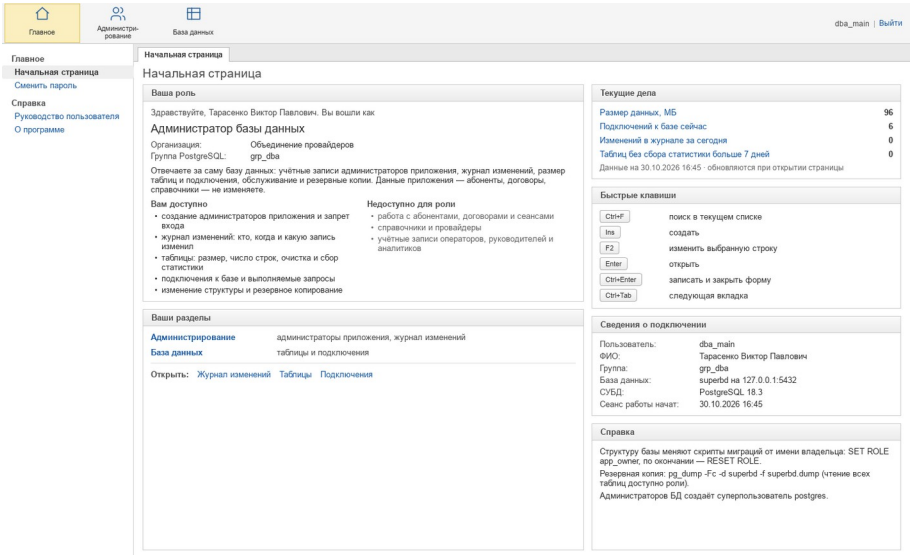


Рисунок 5.8 – Начальная страница администратора БД

Страница «Таблицы базы данных» выводит для каждой таблицы проекта назначение, оценку числа строк, размер с индексами, время последней очистки и

сбора статистики, число последовательных и индексных чтений (функция api.db_tables, рисунок 5.9); страница «Подключения к базе» — текущие сеансы сервера: пользователь, программа, адрес, начало сеанса, состояние и выполняемый запрос (функция api.db_connections, рисунок 5.10).

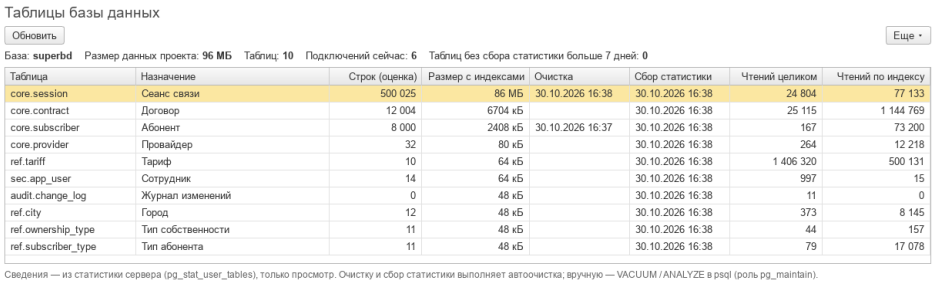


Рисунок 5.9 – Страница «Таблицы базы данных»

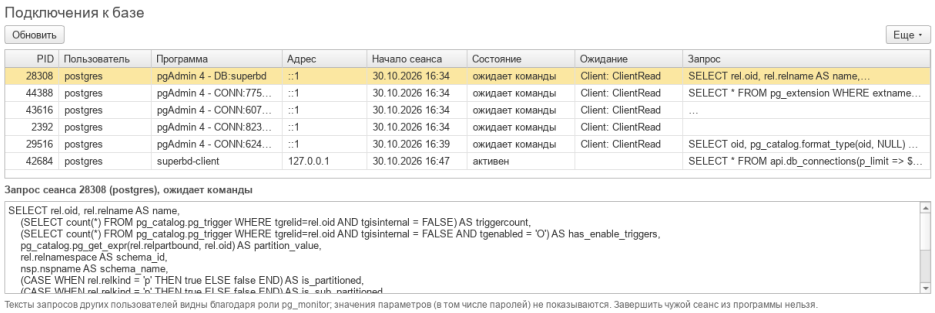


Рисунок 5.10 – Страница «Подключения к базе»

Страница «Журнал изменений» выводит записи журнала за период с отбором по таблице, операции и пользователю и показывает поля записи «было / стало» (функции api.q_change_log и api.q_change_log_fields, рисунок 5.11).

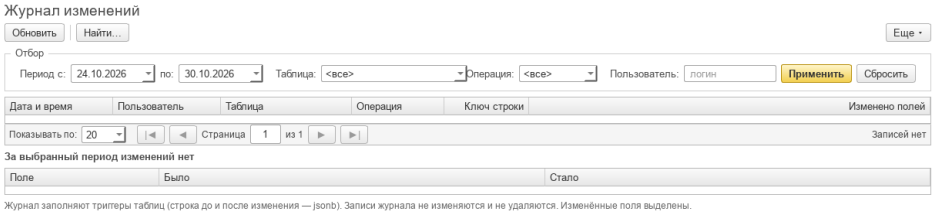


Рисунок 5.11 – Страница «Журнал изменений»

[ЗАГЛУШКА: рисунок журнала будет переснят после реализации триггеров журнала]

5.6 Генерация результатов итоговых запросов (диаграммы, экспорт в Excel)

Раздел «Статистика» руководителя и главного аналитика содержит четыре отчёта, построенные по шаблону отчёта (подраздел 3.3): «Выручка по периодам» (запрос 15, у главного аналитика без отбора — доли провайдеров, запрос 22), «Трафик по тарифам» (запрос 9), «Новые договоры по типам абонентов» (запрос 12) и «Крупнейшие абоненты». В строке параметров задаются период, периодичность (месяц, квартал, год), провайдер (у главного аналитика) и тип абонента — обязательное распределение по временным отрезкам выполняет сервер. Отчёт состоит из табличного документа с итоговой строкой и диаграммы с легендой:

– «Выручка по периодам» — накопительные столбцы «плата за подключение» и «оплата трафика» по периодам (рисунок 4.42); у главного аналитика — столбцы по группам провайдеров: крупнейшие и «Прочие» (рисунок 4.57);

– «Новые договоры по типам абонентов» — линии по типам абонентов (рисунок 4.35);

– «Крупнейшие абоненты» — горизонтальные столбцы выручки первых N абонентов за период;

– «Трафик по тарифам» — таблица использования тарифов, включая неиспользованные (рисунок 4.29).

Команда «Сохранить в Excel…» записывает табличный документ отчёта в файл .xlsx средствами библиотеки QXlsx с заголовком отчёта, параметрами, шапкой таблицы и итоговой строкой; команда «Печать» выводит документ и диаграмму на принтер. Все вычисления выполняет сервер: клиент только раскладывает строки результата по колонкам таблицы и рядам диаграммы.

6 ТЕСТИРОВАНИЕ РАЗРАБОТАННОЙ ИНФОРМАЦИОННОЙ СИСТЕМЫ

Серверная часть проверяется набором автоматических тестов db/tests, который выполняется после каждой пересборки базы данных сценарием db/build_and_test.cmd: база создаётся заново из миграций и данных, затем выполняется db/tests/run_all.sql. Тест выполняет действие от имени нужного пользователя (SET SESSION AUTHORIZATION), сравнивает результат с ожидаемым или с независимым расчётом и выводит строку PASS или FAIL с номером проверки; изменяющие данные тесты выполняются в транзакции с откатом. Состав тестов приведён в таблице 6.1.

Таблица 6.1 – Автоматические тесты серверной части

Группа	**Что проверяется**	**Файл**	**Тестов**
1	Число таблиц, отсутствие своих таблиц пользователей и паролей, комментарии ко всем таблицам и колонкам	010	5
2	Внешние ключи и правила удаления по модели, индексы на всех внешних ключах	010	3
3	Объём и согласованность данных: не менее 10 записей в справочниках и 10 000 в основных таблицах, сеансы в периоде договора, скидка и стоимость сеанса по формуле, тариф по окну времени	010	10
4	Сотрудники: логин — роль PostgreSQL, ровно одна группа, провайдер у операторов и руководителей; триггер проверки логина	010	4
5	Каскадное удаление абонента, запрет удаления тарифа и провайдера, на которые есть ссылки, запрет дубля абонента	010	5
6	Сведения о пользователе, запрет прямого чтения таблиц, отказ пользователю без группы, функции недоступны PUBLIC, счётчики начальной страницы	020	7
7	Запросы 1–10: результат, видимость строк для ролей, постраничный вывод, поиск, экранирование символов шаблона	030	19
8	Отсутствие регулярных выражений, проверки доменов телефона, логина и обозначения тарифа	010	3
9	Общий реестр абонентов: видимость, права на изменение и удаление, оптимистическая блокировка	040	16
10	Разделение администратора приложения и администратора БД: права, создание и блокировка пользователей	050	17
11	Сброс и смена пароля, временный пароль, отказ SB001 до смены пароля	070	8
12	Запросы 11–16, 20, 22 и отчёт «Крупнейшие абоненты»: сравнение с независимым расчётом, итоги, права ролей	060	20
13	Секционирование: устройство секций, каждая строка в своей секции, отсечение секций при планировании и при выполнении, секция договоров нового провайдера, перенос из секции по умолчанию, права администратора БД	080	9
14	Запросы 17–19, 21, 23: сравнение с независимым расчётом, маска, результат без индексного доступа, права ролей, конструкция вида в тексте каждого запроса	090	14
Всего			140

Результат прогона на рабочей базе 30.10.2026 — пройдено 140 из 140 тестов, по файлам — на рисунке 6.1; пересборка базы со всеми данными, включая 500 000 сеансов, занимает около 45 с.

```
== Тесты БД superbd: db\tests\run_all.sql
  010_structure.sql   PASS 30   FAIL 0
  020_whoami.sql   PASS 7   FAIL 0
  030_queries_lr5.sql   PASS 19   FAIL 0
  040_shared_subscribers.sql   PASS 16   FAIL 0
  050_admin_split.sql   PASS 17   FAIL 0
  060_queries_lr6.sql   PASS 20   FAIL 0
  070_password_reset.sql   PASS 8   FAIL 0
  080_partitions.sql   PASS 9   FAIL 0
  090_queries_coursework.sql   PASS 14   FAIL 0
superbd=#
```

Рисунок 6.1 – Итог прогона автоматических тестов по файлам

6.1 Защита от несанкционированного доступа

Проверяется, что пользователь не может получить данные в обход функций api и сверх прав своей группы:

– оператор не читает таблицы схемы core напрямую — ошибка «нет доступа к таблице» (тест 6.2);

– пользователь без группы не получает доступа к схеме api (6.4), функции api и sec не выполняются от имени PUBLIC (6.5), внутреннюю функцию sec.provider_scope оператор вызвать не может (7.18);

– администратору БД закрыты экраны данных абонентов и провайдеров (10.2), администратору приложения — журнал изменений и состояние базы (10.7); администратор приложения не создаёт администраторов (10.10);

– оператор другого провайдера не видит сеансы и договоры чужого провайдера (7.5, 9.8), договор с чужим абонентом по коду не заключить — перебор кодов ничего не даёт (9.5);

– с временным паролем доступны только сведения о себе и смена пароля, остальные функции отказывают с кодом SB001 (11.3); временный пароль не сохраняется ни в одной таблице проекта (11.5);

– ввод служебных символов шаблона «%» и «_» в поиск не выбирает все строки (7.14); значения передаются параметрами, поэтому ввод вида «' OR 1=1 --» обрабатывается как обычная строка поиска.

6.2 Одновременная работа с данными

Изменение абонента и договора выполняется с оптимистической блокировкой: клиент передаёт прочитанную версию строки, функция изменяет строку только при совпадении версии и увеличивает её. Тест 9.11 моделирует двух пользователей: первое изменение проходит (версия 1 → 2), повторное изменение с устаревшей версией отклоняется с кодом 40001 и сообщением «Абонент изменён другим пользователем — обновите карточку». Одновременное создание абонента с тем же наименованием и телефоном не создаёт дубль благодаря уникальному индексу (тесты 5.5, 9.6, 9.10).

6.3 Каскадное удаление

Удаление абонента каскадно удаляет его договоры и сеансы (тест 5.1: договоров 6 → 0, сеансов 68 → 0). Удаление тарифа, на который ссылаются сеансы, и провайдера с договорами отклоняется ограничением RESTRICT (5.3, 5.4). Удалить общего абонента, обслуживаемого несколькими провайдерами, нельзя никому: каскадом были бы удалены чужие договоры (9.16).

6.4 Корректность запросов и отчётов

Результат каждого запроса сравнивается с независимым расчётом другим способом: например, список договоров без сеансов — с расчётом через NOT EXISTS (12.5), выручка за месяц — с прямой суммой операций (12.7), строка «Итого» — с суммой периодов (12.8); число строк объединения — с числом договоров и сеансов (12.9). Проверены также совпадение счётчиков начальной страницы со списками (12.6, 12.10), совпадение результата запроса 19 с

индексным доступом и без него (14.8) и независимость результата от часового пояса клиента (6.7). Секционирование проверяется на рабочей базе: каждая строка лежит в секции своего квартала или провайдера, а секции по умолчанию пусты (13.3), запрос за квартал читает одну секцию сеансов, запрос провайдера — одну секцию договоров (13.4), при общем плане с параметрами лишние секции исключаются при выполнении (13.5), регистрация провайдера создаёт его секцию договоров (13.6).

[ЗАГЛУШКА: проверки триггеров, форм ввода и одновременной работы двух клиентов будут добавлены после реализации]

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработана клиент-серверная информационная система учёта услуг связи интернет-провайдеров (вариант Б-16), в которой логика программного обеспечения размещена на сервере PostgreSQL, а настольное приложение на Qt 6 выполняет функции тонкого клиента.

По задачам, поставленным во введении, получены следующие результаты.

1) Построена концептуальная модель предметной области по методике IDEF1X: восемь сущностей предметной области в третьей нормальной форме и две служебные таблицы; обоснована денормализация — хранение скидки и стоимости сеанса на момент регистрации. Абоненты образуют общий реестр объединения провайдеров, договор является таблицей пересечения провайдера и абонента.

2) Создана база данных из пяти схем с десятью доменами, внешними ключами с выбранными правилами удаления, индексами под условия экранов и явной последовательностью номеров договоров; таблицы заполнены воспроизводимыми псевдослучайными данными — более 520 000 строк. Таблица сеансов секционирована по году и вложенным кварталам, таблица договоров — по провайдерам; секции новых лет и провайдеров создаются функциями и триггером.

3) Реализованы пять групп пользователей средствами PostgreSQL, доступ только через функции и представления схемы api с проверкой группы, ограничение видимости строк провайдером пользователя, создание и блокировка учётных записей из приложения, сброс пароля с временным паролем.

4) Разработаны 23 запроса на выборку всех основных видов: запросы без параметров оформлены представлениями, запросы с параметрами — функциями; запросы 1–16, объединение и подзапрос с CASE являются источниками данных экранов, отчётов и счётчиков клиентского приложения.

Тексты запросов оптимизированы, использование индексов подтверждено планами выполнения.

5) Разработано клиентское приложение: интерфейс по группе пользователя, постраничный вывод, поиск и отбор на сервере, составная форма «абонент — договоры — сеансы», отчёты с диаграммами, выгрузкой в Excel и печатью, страницы администраторов.

6) Корректность серверной части подтверждена 140 автоматическими тестами, в том числе проверками секционирования, защиты от несанкционированного доступа, оптимистической блокировки и каскадного удаления.

Часть требований к системе на момент оформления записки не реализована: триггеры (кроме проверки логина сотрудника) и модифицируемое представление, политики защиты на уровне строк, формы создания и изменения абонентов, договоров, сеансов и справочников в клиенте. Для них в записке оставлены отметки, а порядок реализации определён в документации проекта.

Предложения по развитию системы: перевести ограничение строк с условий в функциях на политики RLS с сохранением функций api как единственной точки доступа, заполнять журнал изменений триггерами, переписать отбор договоров без сеансов с условием на время, чтобы он читал только нужные секции сеансов, добавить загрузку учётных записей сеансов из биллинговых систем провайдеров в формате JSON.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Ильиных, Т. Е. Проектирование реляционных баз данных в нотациях IDEF1X / Т. Е. Ильиных, Л. И. Шустова. — Москва : МИФИ, 2000. — 136 с.

2) Карпова, Т. С. Базы данных: модели, разработка, реализация : учебник для вузов / Т. С. Карпова. — Санкт-Петербург : Питер, 2002. — 304 с.

3) Лузанов, П. В. Postgres. Первое знакомство / П. В. Лузанов, Е. В. Рогов, И. В. Лёвшин. — 12-е изд. — Москва : Постгрес Профессиональный, 2026.

4) Qt 6 Documentation / The Qt Company. — URL: https://doc.qt.io/qt-6/ (дата обращения: 28.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А.
ЛИСТИНГ ШАБЛОНОВ

```
bool ApiCall::select(const
QString &function, const
QList<ApiParam> &params,
QSqlQuery *query, QString
*error)
{
 QStringList names{function};
 QStringList args;
 for (const ApiParam &p :
params) {
   names << p.name;
   args << QStringLiteral("%1 =>
:%1").arg(p.name);
 }
 if (!checkIdentifiers(names,
error))
   return false;
 return
run(QStringLiteral("SELECT *
FROM api.
%1(%2)").arg(function,
args.join(QStringLiteral(", "))),
      params, query, error);
}

bool ApiCall::exec(const QString
&function, const
QList<ApiParam> &params,
QString *error)
{
 QSqlQuery q;
 return select(function, params,
&q, error);
}

bool ApiCall::selectView(const
QString &view, const QString
&orderBy, int limit, int offset,
         QSqlQuery *query,
QString *error)
{
 if (!checkIdentifiers({view,
orderBy}, error))
   return false;
 return
run(QStringLiteral("SELECT *,
count(*) OVER () AS total_count
FROM api.%1 "
            "ORDER BY %2
LIMIT :p_limit
OFFSET :p_offset").arg(view,
orderBy),
      {{QStringLiteral("p_limit"),
qBound(1, limit, 200)},
{QStringLiteral("p_offset"),
qMax(0, offset)}},
      query, error);
}

bool PagedTableModel::reload()
{
 QSqlQuery q;
 bool ok;
 if (m_isView) {
 …
 if (!ok) {
  clear();
  emit failed(m_error);
  return false;
 …
 beginResetModel();
 …
 m_total = 0;
 …
 while (q.next()) {
  …
  for (int i = 0; i < rec.count(); +
+i)
 …
 if (!m_rows.isEmpty()) {
  m_total =
m_rows.first().value(QStringLiter
al("total_count")).toLongLong();
  const QLocale
ru(QLocale::Russian,
QLocale::Russia);
  for (const TotalCell &cell :
m_totalCells) {
…
}

IdleContractsPage::IdleContracts
Page(const Whoami &me,
QWidget *parent)
 : QWidget(parent)
{
 Q_UNUSED(me);
 …
 FormKit::CommandBar
bar(this);
 …
 m_days = new QSpinBox(this);
 …
 m_grid = new DataGrid(this,
true, 20);
  …
  if (id.isValid() && !id.isNull())
    ContractDialog(id,
this).exec();
 …
 connect(open,
&QPushButton::clicked, this,
openCard);
 connect(m_grid,
&DataGrid::activated, this,
openCard);
 connect(refresh,
&QPushButton::clicked, this,
&IdleContractsPage::reload);
 connect(m_days,
&QSpinBox::editingFinished,
this, &IdleContractsPage::reload);
 reload();
}

void
SubscribersPage::subscriberChan
ged(const QVariant &id)
{
 if (!id.isValid() || id.isNull()) {
  m_contractsCaption-
>setText(QStringLiteral("Догово
ры абонента"));
  …
  return;
…
void
SubscribersPage::contractChange
d(const QVariant &id)
 …
 if (!id.isValid() || id.isNull()) {
  …
  return;
                …

FormKit::monthTitle(today)));
…
}
```

```
void ReportPage::buildRevenue()
{
  const QString provider =
providerTitle();

setHeading(QStringLiteral("Выр
учка %1 %2").arg(provider,
byGrain()),
      QStringLiteral("провайдер =
%1; %2").arg(provider,
typeFilter()));
  …
  if (!
ApiCall::select(QStringLiteral("re
port_revenue"), params, &q,
&error)) {
    FormKit::error(this, error);
    return;
          …

QStringLiteral("Подключения,
тыс. руб."),
QStringLiteral("Выручка, тыс.
руб.")});
  …
  while (q.next()) {
    …
    if (!total) {
      categories <<
periodLabel(period.toDate());
  …
  if (!categories.isEmpty())
…
}
```

ПРИЛОЖЕНИЕ Б.

ЛИСТИНГ СЕРВЕРНОГО ПРИЛОЖЕНИЯ

```
\set ON_ERROR_STOP on

DO $$
BEGIN
  IF NOT EXISTS (SELECT 1
FROM pg_catalog.pg_roles
WHERE rolname = 'app_owner')
THEN
    CREATE ROLE app_owner
NOLOGIN NOSUPERUSER
NOCREATEDB CREATEROLE
NOINHERIT;
  END IF;
END $$;
COMMENT ON ROLE
app_owner IS 'Владелец
объектов БД superbd (схемы,
таблицы, функции); входа в
систему не имеет';

SELECT 'CREATE DATABASE
superbd ENCODING "UTF8"
TEMPLATE template0'
WHERE NOT EXISTS
(SELECT 1 FROM
pg_catalog.pg_database WHERE
datname = 'superbd')
\gexec

COMMENT ON DATABASE
superbd IS 'Учёт услуг связи
интернет-провайдеров
(курсовой проект, вариант Б-
16)';

\set ON_ERROR_STOP on
SET client_min_messages =
warning;
\echo 'Сборка БД superbd'

DROP SCHEMA IF EXISTS api,
audit, sec, core, ref CASCADE;

\echo '  миграции'
\ir
migrations/010_schemas_roles_b
ase.sql
\ir migrations/020_domains.sql
\ir migrations/030_ref_tables.sql
\ir migrations/040_core_tables.sql
\ir migrations/042_partitions.sql
\ir
migrations/045_app_user_role_ch
eck.sql
\ir migrations/050_indexes.sql
\ir
migrations/055_sec_whoami.sql
\ir migrations/056_api_home.sql
\ir migrations/057_sec_scope.sql
\ir
migrations/060_queries_simple.s
ql
\ir
migrations/061_queries_aggregat
e.sql
\ir
migrations/062_queries_coursew
ork.sql
\ir
migrations/065_api_lookups.sql
\ir
migrations/066_api_subscriber_w
rite.sql
\ir migrations/070_api_admin.sql
\ir
migrations/075_password_reset.s
ql

\echo '  данные'
\ir seed/010_ref.sql
\ir seed/020_generate.sql
\ir seed/030_demo_users.sql
\ir seed/040_for_queries.sql

ANALYZE;
\echo 'Готово'

DO $$
DECLARE
  g text;
BEGIN
  IF NOT EXISTS (SELECT 1
FROM pg_catalog.pg_roles
WHERE rolname = 'app_owner')
THEN
    CREATE ROLE app_owner
NOLOGIN NOSUPERUSER
NOCREATEDB CREATEROLE
NOINHERIT;
    …
    IF NOT EXISTS (SELECT 1
FROM pg_catalog.pg_roles
WHERE rolname = g) THEN
      EXECUTE format('CREATE
ROLE %I NOLOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE INHERIT',
g);
  …
  EXECUTE format('REVOKE
ALL ON DATABASE %I FROM
PUBLIC', current_database());
  EXECUTE format('GRANT
CONNECT, TEMPORARY ON
DATABASE %I TO grp_admin,
grp_dba, grp_operator,
grp_manager, grp_chief',
  …
  EXECUTE format('GRANT
CREATE ON DATABASE %I
TO app_owner',
current_database());
…
REVOKE CREATE ON
SCHEMA public FROM
PUBLIC;
…
CREATE SCHEMA IF NOT
EXISTS ref   AUTHORIZATION
app_owner;
CREATE SCHEMA IF NOT
EXISTS core
AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT
EXISTS sec
AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT
EXISTS audit
AUTHORIZATION app_owner;
CREATE SCHEMA IF NOT
EXISTS api
AUTHORIZATION app_owner;
…
REVOKE ALL ON SCHEMA
ref, core, sec, audit, api FROM
PUBLIC;
```

```
GRANT USAGE ON SCHEMA api TO grp_admin, grp_dba, grp_operator, grp_manager, grp_chief;

DO $$
DECLARE
  d record;
BEGIN
  FOR d IN
    SELECT * FROM (VALUES
      ('d_name',       'varchar(150)', $c$VALUE = btrim(VALUE) AND VALUE <> ''$c$),
      ('d_address',    'varchar(200)', $c$btrim(VALUE) <> ''$c$),
      ('d_phone',      'varchar(20)',  $c$translate(VALUE, '+0123456789 ()-', '') = '' AND strpos(substr(VALUE, 2), '+') = 0
                       AND length(ltrim(VALUE, '+')) >= 7$c$),
      ('d_year',       'smallint',     $c$VALUE BETWEEN 1970 AND 2100$c$),
      ('d_contract_no', 'varchar(20)', $c$VALUE = btrim(VALUE) AND VALUE <> ''$c$),
      ('d_login',      'varchar(31)',  $c$length(VALUE) >= 3
                       AND translate(left(VALUE, 1), 'abcdefghijklmnopqrstuvwxyz', '') = ''
                       AND translate(VALUE, 'abcdefghijklmnopqrstuvwxyz0123456789_', '') = ''$c$),
      ('d_money',      'numeric(12,2)', $c$VALUE >= 0$c$),
      ('d_price_mb',   'numeric(10,4)', $c$VALUE > 0$c$),
      ('d_percent',    'numeric(5,2)', $c$VALUE BETWEEN 0 AND 100$c$),
      ('d_mb',         'numeric(12,3)', $c$VALUE > 0$c$)
    ) AS t(name, base, chk)
  LOOP
    IF NOT EXISTS (SELECT 1 FROM pg_catalog.pg_type t JOIN pg_catalog.pg_namespace n ON n.oid = t.typnamespace
          WHERE n.nspname = 'ref' AND t.typname = d.name) THEN
      EXECUTE format('CREATE DOMAIN ref.%I AS %s CONSTRAINT %I CHECK (%s)',
          d.name, d.base, 'ck_' || d.name, d.chk);
    END IF;
  END LOOP;
END $$;

CREATE TABLE IF NOT EXISTS core.session (
  id           bigint GENERATED ALWAYS AS IDENTITY,
  started_at   timestamptz   NOT NULL,
  contract_id  integer       NOT NULL,
  provider_id  integer       NOT NULL,
  tariff_id    smallint      NOT NULL,
  volume_mb    ref.d_mb      NOT NULL,
  discount_pct ref.d_percent NOT NULL,
  cost         ref.d_money   NOT NULL,
  CONSTRAINT pk_session PRIMARY KEY (id, started_at),
  CONSTRAINT fk_session_contract_id_provider_id FOREIGN KEY (contract_id, provider_id)
    REFERENCES core.contract (id, provider_id) ON DELETE CASCADE,
  CONSTRAINT fk_session_tariff_id FOREIGN KEY (tariff_id)
    REFERENCES ref.tariff (id) ON DELETE RESTRICT
) PARTITION BY RANGE (started_at);

SET ROLE app_owner;

CREATE INDEX IF NOT EXISTS ix_provider_ownership_type_id ON core.provider (ownership_type_id);
CREATE INDEX IF NOT EXISTS ix_provider_city_id ON core.provider (city_id);
CREATE INDEX IF NOT EXISTS ix_subscriber_city_id ON core.subscriber (city_id);
CREATE INDEX IF NOT EXISTS ix_subscriber_subscriber_type_id ON core.subscriber (subscriber_type_id);
CREATE INDEX IF NOT EXISTS ix_session_tariff_id ON core.session (tariff_id);
CREATE INDEX IF NOT EXISTS ix_app_user_provider_id ON sec.app_user (provider_id);
…
CREATE INDEX IF NOT EXISTS ix_session_contract_id_started_at ON core.session (contract_id, started_at);
CREATE INDEX IF NOT EXISTS ix_session_started_at ON core.session (started_at);
CREATE INDEX IF NOT EXISTS ix_contract_provider_id_connected_on ON core.contract (provider_id, connected_on);
CREATE INDEX IF NOT EXISTS ix_change_log_changed_at ON audit.change_log (changed_at);
…
CREATE INDEX IF NOT EXISTS ix_contract_subscriber_id_provider_id ON core.contract (subscriber_id, provider_id);
…
CREATE UNIQUE INDEX IF NOT EXISTS uq_subscriber_lower_name_phone_digits
…
```

```
CREATE UNIQUE INDEX IF
NOT EXISTS
uq_contract_provider_id_ip_addr
ess_active
…
RESET ROLE;

CREATE OR REPLACE
FUNCTION
sec.scope_data(p_provider_id
integer DEFAULT NULL)
RETURNS integer[]
LANGUAGE sql STABLE
SECURITY DEFINER
SET search_path = pg_catalog,
pg_temp
AS $$
  SELECT
sec.provider_scope(sec.sees_all_d
ata(), p_provider_id)
$$;

CREATE OR REPLACE
FUNCTION api.report_revenue(
  p_from date,
…
RETURNS TABLE (
…
LANGUAGE plpgsql STABLE
SECURITY DEFINER
SET search_path = pg_catalog,
pg_temp
SET TimeZone =
'Europe/Moscow'
SET plan_cache_mode =
force_custom_plan
AS $$
  …
  RETURN QUERY
  SELECT k.period, CASE
WHEN k.is_total THEN 'Итого'
ELSE sec.period_title(p_grain,
k.period) END,
     k.new_contracts,
k.volume_mb, k.connection_fees,
k.revenue - k.connection_fees,
k.revenue
  FROM (SELECT
date_trunc(p_grain, r.at)::date AS
period,

grouping(date_trunc(p_grain,
r.at)::date) = 1                AS
is_total,
       count(*) FILTER (WHERE
r.kind = 'подключение')
AS new_contracts,
       coalesce(sum(r.volume_mb),
0)                              AS
volume_mb,
       coalesce(sum(r.amount)
FILTER (WHERE r.kind =
'подключение'), 0)  AS
connection_fees,
       coalesce(sum(r.amount), 0)
AS revenue
     FROM api.v_revenue_items r
     WHERE r.at >= p_from AND
r.at < p_to + 1
      AND (p_provider_id IS
NULL OR r.provider_id =
p_provider_id)
      AND (p_subscriber_type_id
IS NULL OR
r.subscriber_type_id =
p_subscriber_type_id)
     GROUP BY ROLLUP
(date_trunc(p_grain, r.at)::date)) k
  ORDER BY k.is_total, k.period;
END $$;

CREATE OR REPLACE
FUNCTION api.contract_create(
  p_subscriber_id integer,
…
RETURNS TABLE (contract_id
integer, contract_no text,
subscriber_id integer,
subscriber_created boolean)
LANGUAGE plpgsql
VOLATILE SECURITY
DEFINER
SET search_path = pg_catalog,
pg_temp
SET TimeZone =
'Europe/Moscow'
AS $$
    …
    SELECT s.id, s.address INTO
v_sub, v_sub_address
    FROM core.subscriber s
    WHERE s.id = p_subscriber_id
     AND EXISTS (SELECT 1
FROM core.contract c WHERE
c.subscriber_id = s.id AND
c.provider_id = v_provider)
    FOR KEY SHARE;
    IF NOT FOUND THEN
      RAISE EXCEPTION
'Абонент не найден среди
абонентов провайдера' USING
ERRCODE = '22023';
    END IF;
  ELSE
    IF coalesce(btrim(p_name), '')
= '' OR coalesce(btrim(p_phone),
'') = '' THEN
      RAISE EXCEPTION
'Укажите наименование и
телефон абонента' USING
ERRCODE = '22023';
    END IF;
    SELECT s.id, s.address INTO
v_sub, v_sub_address
    FROM core.subscriber s
    WHERE lower(s.name) =
lower(btrim(p_name))
     AND translate(s.phone, '+ ()-',
'') = translate(btrim(p_phone), '+
()-', '')
    FOR KEY SHARE;
    IF NOT FOUND THEN
      BEGIN
  …
  END;
END $$;

CREATE OR REPLACE
FUNCTION
api.user_reset_password(p_login
text)
RETURNS TABLE
(temp_password text, valid_until
timestamptz)
LANGUAGE plpgsql
VOLATILE SECURITY
DEFINER
SET search_path = pg_catalog,
pg_temp
AS $$
DECLARE
  v_group text :=
sec.current_group();
  v_target text :=
sec.group_of(p_login::name);
  v_password text;
BEGIN
  PERFORM
sec.require_group(ARRAY['grp_
admin', 'grp_dba']);
  IF p_login = session_user THEN
```

```
    RAISE EXCEPTION 'Свой пароль меняют через «Сменить пароль»' USING ERRCODE = '22023';
  END IF;
  IF v_target IS NULL OR NOT v_target = ANY (sec.manageable_groups(v_group))
    OR NOT EXISTS (SELECT 1 FROM sec.app_user u WHERE u.login = p_login) THEN
    RAISE EXCEPTION 'Нет прав на учётную запись «%»', p_login USING ERRCODE = '42501';
  END IF;
  v_password := sec.issue_temp_password(p_login);
  RETURN QUERY SELECT v_password, r.rolvaliduntil FROM pg_catalog.pg_roles r WHERE r.rolname = p_login;
END $$;

WITH c AS (
    SELECT ct.id, ct.provider_id, ct.price_per_mb, ct.connected_on,
       least(coalesce(ct.closed_on + 1, current_date), current_date) AS until_on,
       least(coalesce(ct.closed_on + 1, current_date), current_date) - ct.connected_on AS days,
    …
    FROM core.contract ct
    JOIN core.subscriber sb ON sb.id = ct.subscriber_id
    JOIN ref.subscriber_type st ON st.id = sb.subscriber_type_id),
     …
     FROM w),
    …
    SELECT n.id AS contract_id, n.provider_id, n.price_per_mb, n.subscriber_type,
    n.connected_on, n.until_on, g
    FROM n CROSS JOIN LATERAL generate_series(1, n.n) g
    ORDER BY n.id, g),
    …
    SELECT p.contract_id, p.provider_id, p.price_per_mb, p.subscriber_type,
    …
    FROM pick p),
    …
    SELECT s.*,
      CASE WHEN s.subscriber_type IN ('Школа', 'Колледж', 'Вуз') AND s.manual_roll < 0.40
           …
           WHERE tr.time_from IS NOT NULL
    …
    FROM s)
INSERT INTO core.session (started_at, contract_id, provider_id, tariff_id, volume_mb, discount_pct, cost)
SELECT t.started_at, t.contract_id, t.provider_id, t.tariff_id, t.volume_mb, tr.discount_pct,
…
FROM t
JOIN ref.tariff tr ON tr.id = t.tariff_id
ORDER BY t.started_at;

SET client_min_messages = warning;
DROP TABLE IF EXISTS pg_temp.test_result;
SET client_min_messages = notice;
CREATE TEMP TABLE test_result (
  n      serial,
  name   text    NOT NULL,
  ok     boolean NOT NULL,
  detail text
);

CREATE OR REPLACE FUNCTION pg_temp.check(p_name text, p_ok boolean, p_detail text DEFAULT NULL)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO pg_temp.test_result (name, ok, detail) VALUES (p_name, coalesce(p_ok, false), p_detail);
  RAISE NOTICE '% %', CASE WHEN coalesce(p_ok, false) THEN 'PASS' ELSE 'FAIL' END,
    p_name || coalesce(' — ' || p_detail, '');
END $$;
```

ПРИЛОЖЕНИЕ В.

ЛИСТИНГ КЛИЕНТСКОГО ПРИЛОЖЕНИЯ

```
int main(int argc, char *argv[])
{
  QApplication app(argc, argv);

QApplication::setApplicationNa
me(QStringLiteral("СуперБД"));

QApplication::setWindowIcon(QI
con(QStringLiteral(":/icons/app.s
vg")));

QLocale::setDefault(QLocale(QL
ocale::Russian,
QLocale::Russia));
  applyStyle(app);
  …
  if (!
QSqlDatabase::isDriverAvailable
(QStringLiteral("QPSQL"))) {
    …
    return 1;
  …
  for (;;) {
    …
    if (!
dev.screenshotLogin.isEmpty()) {
      LoginDialog dialog(config);
      …
      if (!dev.loginError.isEmpty())
      …
      scheduleScreenshot(&dialog,
dev.screenshotLogin, 300);
      return app.exec();
    …
    if (!dev.devAs.isEmpty()) {
      if (!
Db::open(config.connection,
dev.devUser, QString(), &error)
        …

QMessageBox::critical(nullptr,
QStringLiteral("СуперБД"),
error);
        return 1;
      …
      LoginDialog dialog(config);
      if (dialog.exec() !=
QDialog::Accepted)
        return 0;
      …
      if (!Db::loadWhoami(&me,
&error)) {

QMessageBox::critical(nullptr,
QStringLiteral("СуперБД"),
error);
        Db::close();
    …
    if (me.mustChangePassword) {
      if (me.groupName ==
QLatin1String("grp_dba")) {
…
}

bool Db::open(const
ConnectionParams &params,
const QString &user, const
QString &password, QString
*error)
{
  close();
  QSqlDatabase db =
QSqlDatabase::addDatabase(QStr
ingLiteral("QPSQL"),
kConnection);
  db.setHostName(params.host);
  db.setPort(params.port);

db.setDatabaseName(params.data
base);
  db.setUserName(user);
  db.setPassword(password);

db.setConnectOptions(QStringLit
eral("connect_timeout=5;applicati
on_name=superbd-client"));
  if (!db.open()) {
    if (error)
      *error =
userMessage(db.lastError());
    close();
    return false;
  }
  return true;
}

QString Db::userMessage(const
QSqlError &error)
{
  QString text =
error.databaseText().trimmed();
  if (text.isEmpty())
    text =
error.driverText().trimmed();
  text =
text.section(QLatin1Char('\n'), 0,
0);
  static const QStringList levels =
{QStringLiteral("ОШИБКА:"),
QStringLiteral("ВАЖНО:"),
QStringLiteral("ПАНИКА:"),

QStringLiteral("ERROR:"),
QStringLiteral("FATAL:"),
QStringLiteral("PANIC:")};
  for (const QString &level :
levels) {
    if (text.startsWith(level)) {
      text =
text.mid(level.size()).trimmed();
      break;
    }
  }
  if
(text.endsWith(QLatin1Char(')'))
&& text.size() >= 7 &&
text.at(text.size() - 7) ==
QLatin1Char('(')) {
    const QStringView code =
QStringView(text).sliced(text.size
() - 6, 5);
    bool isCode = true;
    for (const QChar ch : code)
      isCode = isCode && ((ch >=
QLatin1Char('0') && ch <=
QLatin1Char('9')) || (ch >=
QLatin1Char('A') && ch <=
QLatin1Char('Z')));
    if (isCode)
      text = text.left(text.size() -
7).trimmed();
  }
  return text.isEmpty() ?
QStringLiteral("Неизвестная
ошибка базы данных") : text;
}
```

```
QList<Section>
RoleConfig::sectionsFor(const
QString &group)
{
  const QString subscribersTitle =
QStringLiteral("Абоненты и\
ндоговоры");
  if (group ==
QLatin1String("grp_operator"))
    return {home(),
  …
  if (group ==
QLatin1String("grp_manager"))
    return {home(),
  …
  if (group ==
QLatin1String("grp_chief"))
    return {home(),
  …
  if (group ==
QLatin1String("grp_admin"))
    return {home(),
  …
  if (group ==
QLatin1String("grp_dba"))
    return {home(),
  …
  return {home()};
}

void LoginDialog::tryLogin()
{
  const QString user = ui-
>userEdit->text().trimmed();
  if (user.isEmpty()) {
    QMessageBox::warning(this,
windowTitle(),
QStringLiteral("Укажите
пользователя."));
    …
    return;
  …
  if (!Db::open(params, user, ui-
>passwordEdit->text(), &error)
|| !Db::loadWhoami(&me,
&error)) {
    Db::close();
    showError(error);
    …
    return;
  …
  hideError();
  …
  accept();
}

void UsersPage::resetPassword()
{
  const QString login = m_users-
>currentValue(QStringLiteral("lo
gin")).toString();
  if (login.isEmpty())
    return;
  const QString fullName =
m_users-
>currentValue(QStringLiteral("fu
ll_name")).toString();
  ResetConfirmDialog
confirm(login, fullName,
m_users-
>currentValue(QStringLiteral("gr
oup_title")).toString(),
              m_users-
>currentValue(QStringLiteral("pr
ovider_name")).toString(), this);
  if (confirm.exec() !=
QDialog::Accepted)
    return;
  QSqlQuery q;
  QString error;
  if (!
ApiCall::select(QStringLiteral("u
ser_reset_password"),
{{QStringLiteral("p_login"),
login}}, &q, &error)
    || !q.next()) {
    FormKit::error(this,
error.isEmpty() ?
QStringLiteral("Сервер не
вернул временный пароль") :
error);
    return;
  }
  QString password =
q.value(0).toString();
  const QDateTime validUntil =
q.value(1).toDateTime();
  q = QSqlQuery();
  m_users->reload();

TempPasswordDialog::showOnce
(this, login, fullName, password,
validUntil);
}

void
PasswordChangeDialog::save()
{
  if (m_new->text() != m_repeat-
>text()) {
    FormKit::error(this,
QStringLiteral("Пароли не
совпадают."));
    m_repeat->setFocus();
    return;
  }
  QString error;
  if (!
ApiCall::exec(QStringLiteral("ch
ange_own_password"),
{{QStringLiteral("p_new"),
m_new->text()}}, &error)) {
    FormKit::error(this, error);
    m_new->setFocus();
    m_new->selectAll();
    return;
  }
  m_new->clear();
  m_repeat->clear();
  accept();
}

void ReportChart::setData(Kind
kind, const QString &title, const
QStringList &categories, const
QList<Series> &series)
{
  auto *chart = new QChart;
  chart->setTitle(title);
  …
  styleAxis(axisValues);
  …
  styleAxis(axisCategories);
  …
  if (kind == Lines) {
    …
    for (const Series &s : series) {
      …
      QPen pen(s.color);
      …
      for (int i = 0; i <
s.values.size(); ++i) {
    …
    if (kind == HorizontalBars)
    …
    QList<double>
totals(categories.size(), 0.0);
    for (const Series &s : series) {
      …
      for (int i = 0; i <
categories.size(); ++i) {
    …
    if (kind == HorizontalBars) {
  …
  fillLegend(series);
```

```
}

bool save(QWidget *parent, const
QString &suggestedName, const
QStringList &heading, const
QAbstractItemModel *model)
{
  const QString dir =
QStandardPaths::writableLocatio
n(QStandardPaths::DocumentsLo
cation);
  QString path =
QFileDialog::getSaveFileName(p
arent, QStringLiteral("Сохранить
в Excel"),
                    …

QStringLiteral("Книга Excel
(*.xlsx)"));
  if (path.isEmpty())
    return false;
  if (!
path.endsWith(QStringLiteral(".xl
sx"), Qt::CaseInsensitive))
  …
  for (int i = 0; i < heading.size();
++i)
  …
  for (int c = 0; c < columns; ++c)
{
  …
  for (int r = 0; r < model-
>rowCount(); ++r, ++row) {
    …
    for (int c = 0; c < columns; +
+c) {
      …
      if (!v.isValid())
      …
      if (v.typeId() ==
QMetaType::Double)
  …
  if (!xlsx.saveAs(path)) {
    FormKit::error(parent,
QStringLiteral("Не удалось
сохранить файл
%1").arg(QDir::toNativeSeparato
rs(path)));
    return false;
  …
  FormKit::status(parent,
QStringLiteral("Отчёт сохранён:
%1").arg(QDir::toNativeSeparato
rs(path)));
  return true;
}
```

```
cmake_minimum_required(VERS
ION 3.21)
project(superbd_client VERSION
0.1 LANGUAGES CXX)

# Тонкий клиент СуперБД: Qt 6
Widgets + QPSQL. Интерфейс
— строго по макету docs/ui/ (ui-
spec.md).
set(CMAKE_CXX_STANDARD
17)
set(CMAKE_CXX_STANDARD
_REQUIRED ON)
…
find_package(Qt6 6.5
REQUIRED COMPONENTS
Core Gui Widgets Sql Svg Charts
PrintSupport)
…
find_package(QXlsxQt6
REQUIRED)
qt_standard_project_setup()
…
set(CMAKE_AUTOUIC_SEAR
CH_PATHS $
{CMAKE_CURRENT_SOURCE
_DIR}/src/ui $
{CMAKE_CURRENT_SOURCE
_DIR}/src/ui/pages)
…
option(SUPERBD_DEV_TOOLS
"Ключи --dev-as и --screenshot
для разработки" OFF)
…
target_include_directories(superb
d PRIVATE src)
…
target_include_directories(superb
d PRIVATE src/export/compat)
…
if(SUPERBD_DEV_TOOLS)

target_compile_definitions(super
bd PRIVATE
SUPERBD_DEV_TOOLS)
endif()
```

ПРИЛОЖЕНИЕ Г.
РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Г.1 Назначение программы и требования

Программа «СуперБД» предназначена для учёта услуг связи интернет-провайдеров объединения: просмотра абонентов, договоров и сеансов связи, анализа выручки и трафика. Для работы нужен компьютер с Windows 10 или 11, папка программы (исполняемый файл superbd.exe с библиотеками Qt и PostgreSQL) и учётная запись, выданная администратором приложения. Программа подключается к серверу базы данных объединения по сети.

Г.2 Вход в программу

При запуске открывается окно «Доступ к информационной базе» (рисунок 3.3). Введите имя пользователя (логин) и пароль, выданные администратором. Поля «Сервер», «Порт» и «База данных» (кнопка «Параметры подключения») сообщает администратор; пароль программа не сохраняет. Нажмите «ОК».

Если вход не удался, под полями выводится причина: неверный пароль, запрещённый вход или истёкший срок временного пароля. После сброса пароля администратором первый вход выполняется с временным паролем, и программа сразу открывает форму «Смена пароля» (рисунок 5.6): введите новый пароль дважды — не короче 8 символов и не совпадающий с логином — и нажмите «Сохранить и продолжить». Свой пароль можно сменить и позже по ссылке «Сменить пароль» начальной страницы.

Г.3 Главное окно

Главное окно состоит из панели разделов слева, панели навигации раздела, вкладок открытых форм и строки состояния (рисунок 3.4). Состав разделов зависит от роли: оператор работает с разделами «Абоненты и договоры», «Сеансы связи», «Мой провайдер»; руководитель — дополнительно со «Статистикой»; главный аналитик — с разделами «Провайдеры», «Абоненты и

договоры», «Сеансы связи», «Статистика». Руководитель и главный аналитик только просматривают данные — в заголовке форм выводится пометка «(только просмотр)».

Начальная страница открыта всегда; щелчок по счётчику блока «Текущие дела» открывает соответствующий список или отчёт.

Во всех списках действуют общие правила:

– список выводится страницами; размер страницы (20, 100 или 200 записей) выбирается в поле «Показывать по», переход между страницами — кнопками и полем «Страница N из M» под таблицей;

– поиск выполняется по полю «Поиск» командной панели (Ctrl+F): по началу наименования, номера договора или цифрам телефона в любом написании, Enter — найти, «Отменить поиск» — показать всё;

– двойной щелчок по строке или Enter открывает карточку записи;

– итоговая строка «Итого…» считается сервером по всем отобранным записям, а не только по видимой странице.

Г.4 Работа оператора

Раздел «Абоненты и договоры», форма «Абоненты» (рисунок 5.1). Верхняя таблица — абоненты, у которых есть договор с вашим провайдером. Отбор задаётся полями «Тип абонента» и флажком «Только с действующими договорами». При выборе абонента в средней таблице выводятся его договоры с вашим провайдером, при выборе договора в нижней — сеансы договора с итогами. Двойной щелчок по абоненту открывает карточку абонента, по договору — карточку договора с вкладкой «Сеансы».

Форма «Договоры» выводит договоры, заключённые в периоде: задайте даты, при необходимости абонента и флажок «Только действующие», нажмите «Применить».

Раздел «Сеансы связи», форма «Журнал сеансов» (рисунок 4.17): задайте период, при необходимости договор, IP-адрес или подсеть (например,

10.200.0.0/24) и тариф, нажмите «Применить». Строка «Итого за период» показывает число сеансов, объём и стоимость по всему отбору.

Раздел «Мой провайдер» показывает карточку вашего провайдера и список сотрудников. Список «Договоры без сеансов» открывается счётчиком начальной страницы; поле «Без сеансов более … дней» меняет границу простоя.

[ЗАГЛУШКА: создание абонента и договора, регистрация сеанса, изменение и расторжение договора будут описаны после реализации форм]

Г.5 Работа руководителя провайдера и главного аналитика

Руководитель видит данные своего провайдера, главный аналитик — данные всех провайдеров; формы «Абоненты», «Договоры» и «Журнал сеансов» работают так же, как у оператора, без команд изменения. Главный аналитик дополнительно работает с разделом «Провайдеры»: список провайдеров с числом действующих договоров и договоров, заключённых в периоде, и сотрудники выбранного провайдера.

Раздел «Статистика» содержит отчёты «Выручка по периодам», «Трафик по тарифам», «Новые договоры по типам абонентов» и «Крупнейшие абоненты» (рисунок 4.42). Порядок работы с отчётом:

1) Задайте период (поля дат или меню «…») и периодичность — месяц, квартал или год; главный аналитик может выбрать провайдера.

2) Нажмите «Сформировать»: выводится табличный документ с итоговой строкой и диаграмма с легендой.

3) Для выгрузки нажмите «Сохранить в Excel…» и укажите имя файла; для печати — «Печать».

Г.6 Сообщения программы

Если действие недоступно вашей роли, выводится сообщение «Недостаточно прав для этого действия». При потере связи с сервером войдите в программу повторно; если ошибка повторяется, обратитесь к администратору (блок «Справка» начальной страницы).

ПРИЛОЖЕНИЕ Д.
РУКОВОДСТВО СУПЕРПОЛЬЗОВАТЕЛЯ

Д.1 Требования к серверу

Сервер базы данных — PostgreSQL 18 (допустима версия 16 и выше) на Windows или Linux, кодировка базы UTF-8. Рекомендуемые параметры сервера приведены в разделе 2 (таблица 2.1); для защиты паролей в журнале сервера установите log_parameter_max_length = 0. Нужны программы psql и pgAdmin 4; пароль суперпользователя хранится в файле %APPDATA%\postgresql\ pgpass.conf, а не в сценариях.

Д.2 Развёртывание базы данных

1) Создайте базу и роль-владельца (один раз, суперпользователем из базы postgres): psql -U postgres -d postgres -v ON_ERROR_STOP=1 -f db/create_db.sql

2) Создайте объекты и данные (повторный запуск пересоздаёт схемы проекта вместе с данными; группы и учётные записи пользователей сохраняются): psql -U postgres -d superbd -v ON_ERROR_STOP=1 -f db/build.sql

3) Проверьте базу автоматическими тестами (в конце выводится «Тесты: пройдено 140 из 140»): psql -U postgres -d superbd -v ON_ERROR_STOP=1 -f db/tests/run_all.sql

На Windows шаги 2 и 3 выполняет сценарий db\build_and_test.cmd: полный вывод записывается в db\build_and_test.log, в окне выводятся результаты тестов и итог (рисунок 6.1).

Демонстрационные учётные записи (op_donnet, mgr_donnet, chief_main, adm_main, dba_main) создаются сценарием db/seed/030_demo_users.sql без паролей; пароль задаётся командой \password <логин> в psql.

Д.3 Учётные записи администраторов

Первого администратора БД создаёт суперпользователь по образцу сценария db/seed/030_demo_users.sql: роль с правом входа, членство в группе

grp_dba и строка таблицы «Сотрудник» с ФИО; пароль задаётся командой \ password <логин>. В разделе «Администрирование» → «Пользователи» администратор БД создаёт администраторов приложения (с временным паролем на сутки), запрещает и разрешает им вход, сбрасывает пароль; свой пароль меняет командой \password в psql (на сервер передаётся только хеш).

Для изменения структуры базы администратор БД выполняет SET ROLE app_owner.

Д.4 Работа в программе

Раздел «База данных» содержит формы «Таблицы» и «Подключения»:

– «Таблицы базы данных» (рисунок 5.9) — назначение, число строк, размер, последняя очистка и сбор статистики каждой таблицы проекта; статистику собирает команда ANALYZE, очистку — VACUUM (право даёт встроенная роль pg_maintain);

– «Подключения к базе» (рисунок 5.10) — текущие сеансы: пользователь, программа, адрес, начало сеанса, состояние и текущий запрос.

Форма «Журнал изменений» выводит записи журнала за период с отбором по таблице, операции и пользователю и изменённые поля записи «было / стало».

Д.5 Резервное копирование и восстановление

Администратор БД входит во встроенную роль pg_read_all_data и может выполнять резервное копирование без прав суперпользователя:

pg_dump -U <логин администратора БД> -h <сервер> -d superbd -Fc -f superbd_<дата>.backup

Копия в формате custom (-Fc) содержит структуру и данные всех схем проекта (около 17 МБ). Восстановление выполняет суперпользователь в пустую базу, созданную сценарием create_db.sql:

pg_restore -U postgres -d superbd --clean --if-exists superbd_<дата>.backup

Учётные записи пользователей в копию базы не входят: их сохраняет суперпользователь командой pg_dumpall --roles-only.

ПРИЛОЖЕНИЕ Е.
РУКОВОДСТВО АДМИНИСТРАТОРА

Е.1 Назначение роли

Администратор приложения ведёт учётные записи операторов, руководителей провайдеров и главных аналитиков, справочники, провайдеров и сведения общих абонентов. Абонентов, договоров и сеансов администратор не видит. Учётную запись администратора приложения создаёт администратор БД (приложение Д). Начальная страница администратора содержит счётчики «Пароли истекают до …», «Пользователи с запрещённым входом», «Провайдеры без сотрудников», «Пользователей системы» (рисунок 5.7).

Е.2 Пользователи

Раздел «Администрирование» → «Пользователи» (рисунок 4.33): слева — группы с числом пользователей, справа — пользователи выбранной группы (логин, ФИО, группа, провайдер, «Вход разрешён», «Пароль действует до»; у временного пароля — пометка «(временный)»).

Создание пользователя:

1) Нажмите «Создать» — откроется форма «Пользователь (создание)».

2) Введите логин (латинские строчные буквы, цифры и «_», от 3 до 31 символа, начинается с буквы) и ФИО.

3) Выберите группу: оператор, руководитель провайдера или главный аналитик. Для оператора и руководителя выберите провайдера кнопкой «…»; главному аналитику провайдер не указывается.

4) При необходимости снимите флажок «Вход разрешён» и нажмите «Записать и закрыть».

5) Программа покажет временный пароль, действующий сутки: кнопка «Копировать» помещает его в буфер обмена. Пароль показывается один раз — передайте его сотруднику; при первом входе сотрудник задаст свой пароль.

Команда «Запретить вход» блокирует учётную запись выбранного пользователя (данные сохраняются), «Разрешить вход» — снимает блокировку. Собственную учётную запись и учётные записи администраторов заблокировать нельзя.

Сброс пароля: выберите пользователя и нажмите «Сбросить пароль…», подтвердите сброс (рисунок 5.5). Сервер выдаёт новый временный пароль на сутки; до его смены сотрудник может только сменить пароль. Себе сбросить пароль нельзя — свой пароль меняется по ссылке «Сменить пароль» начальной страницы.

Е.3 Провайдеры

Раздел «Провайдеры» выводит всех провайдеров объединения с типом собственности, городом, адресом, телефоном, годом начала работы и числом договоров; в нижней таблице — сотрудники выбранного провайдера с группой и признаком входа. Флажок «Без сотрудников» (его включает и счётчик начальной страницы) оставляет провайдеров, которым ещё не назначены операторы и руководители (рисунок 4.44); назначьте им сотрудников в разделе «Пользователи».

Е.4 Справочники и общие абоненты

Список «Общие абоненты» раздела «Справочники» выводит абонентов, у которых есть договоры не меньше чем с заданным числом провайдеров (поле «Провайдеров не меньше»). Сведения таких абонентов (наименование, тип, город, адрес, телефон) меняет только администратор приложения: оператор может менять лишь абонента, которого не обслуживают другие провайдеры.

[ЗАГЛУШКА: изменение и удаление пользователей, создание и изменение провайдеров, работа со справочниками и общими абонентами будут описаны после реализации форм]